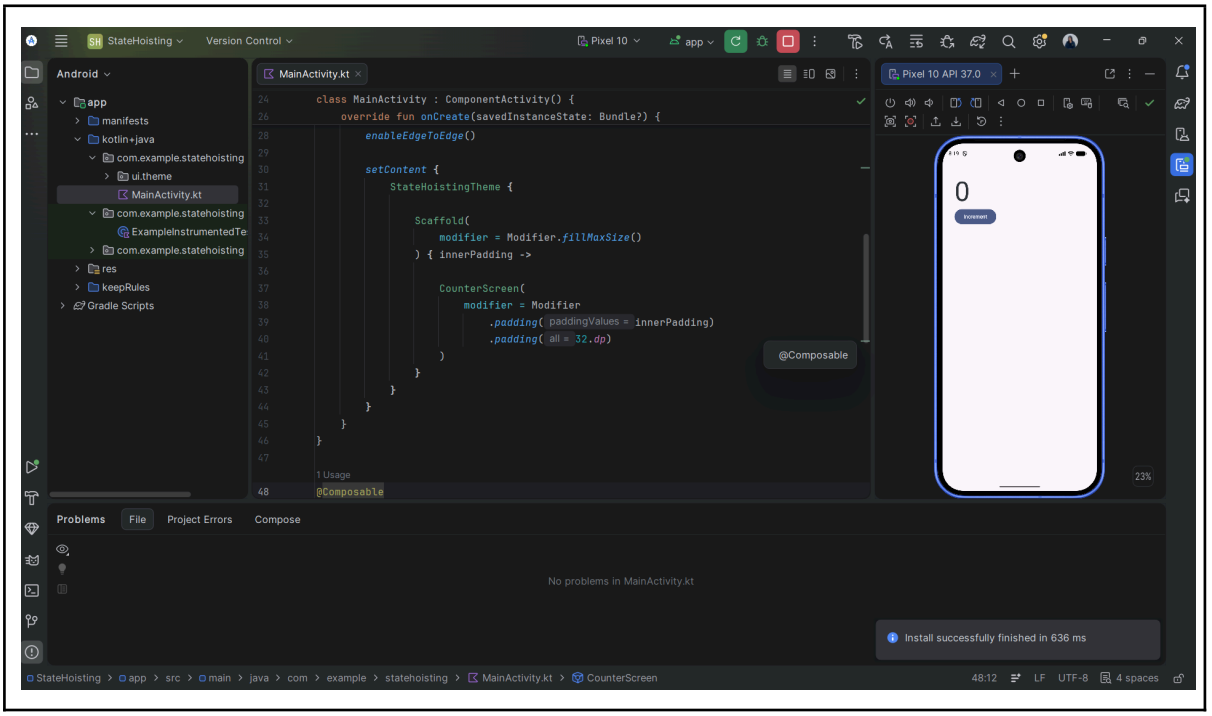
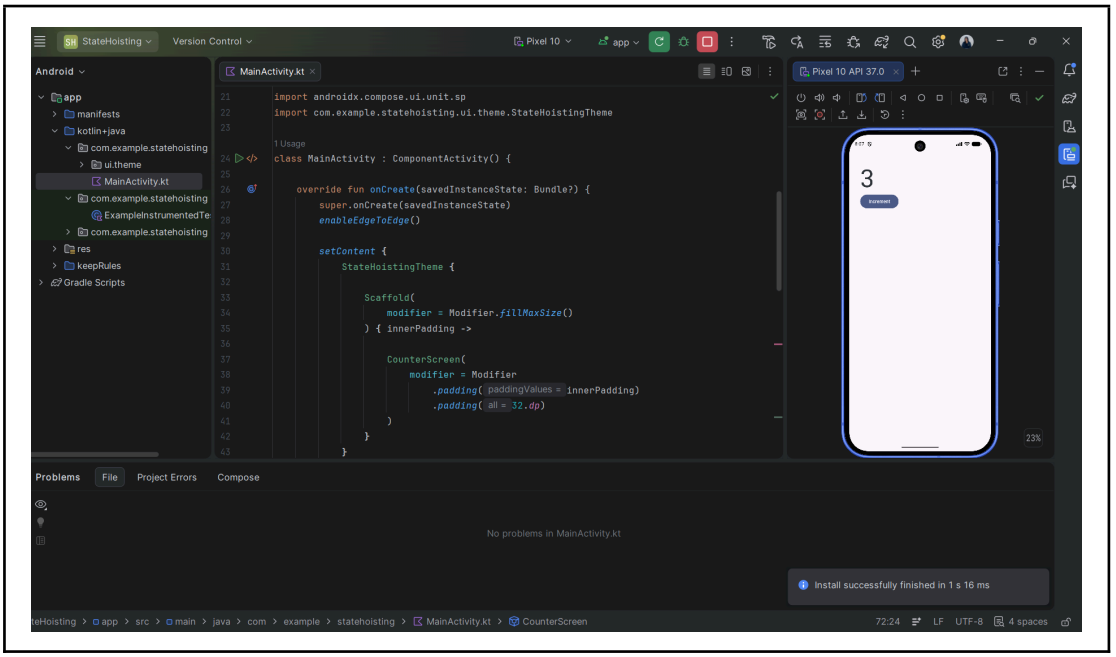
Nama : Intan Nur Azizah
NIM : 215150600111005
Kelas : PAPB - PTI A

Percobaan 1 (4.5.1 State Hoisting)



1. Setelah memencet tombol increment



b. Setiap kali tombol Increment diklik, angka yang ditampilkan pada Text akan bertambah 1. Misalnya dari 0 menjadi 1, lalu 2, 3, dan seterusnya. Hal ini terjadi karena event klik menjalankan number++.
c. Hal tersebut terjadi karena variabel number merupakan state yang dibuat menggunakan:

```
var number by remember { mutableStateOf(0) }
```

Ketika tombol diklik, kode number++ mengubah nilai number. Karena number adalah state, perubahan nilainya membuat Compose memperbarui tampilan Text yang menggunakan nilai tersebut.

d. Variabel state yang berperan adalah number, yang dideklarasikan dengan

```
var number by remember { mutableStateOf(0) }
```

Nilai number kemudian ditampilkan melalui:

```
Text(text = "$number", fontSize = 72.sp)
```

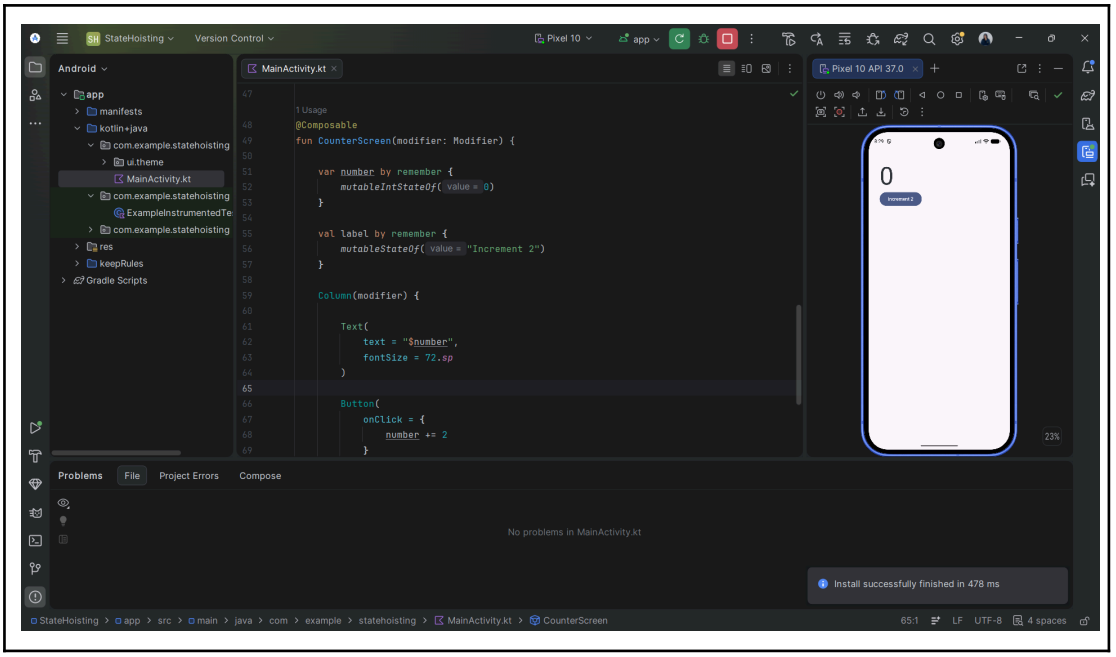
e. Variabel state yang digunakan adalah label, yang dideklarasikan dengan

```
val label by remember { mutableStateOf("Increment")
}
```

Kemudian nilainya ditampilkan di dalam Button melalui:

```
Text(text = "$label")
```

2. Setelah mengubah kode



b. Perubahan dilakukan di dalam fungsi

```
@Composable
fun CounterScreen(...)
```

Perubahan pertama dilakukan di

```
val label by remember { mutableStateOf("Increment")
}
```

menjadi

```
val label by remember { mutableStateOf("Increment
2") }
```

Kedua, perubahan dilakukan pada event tombol dari

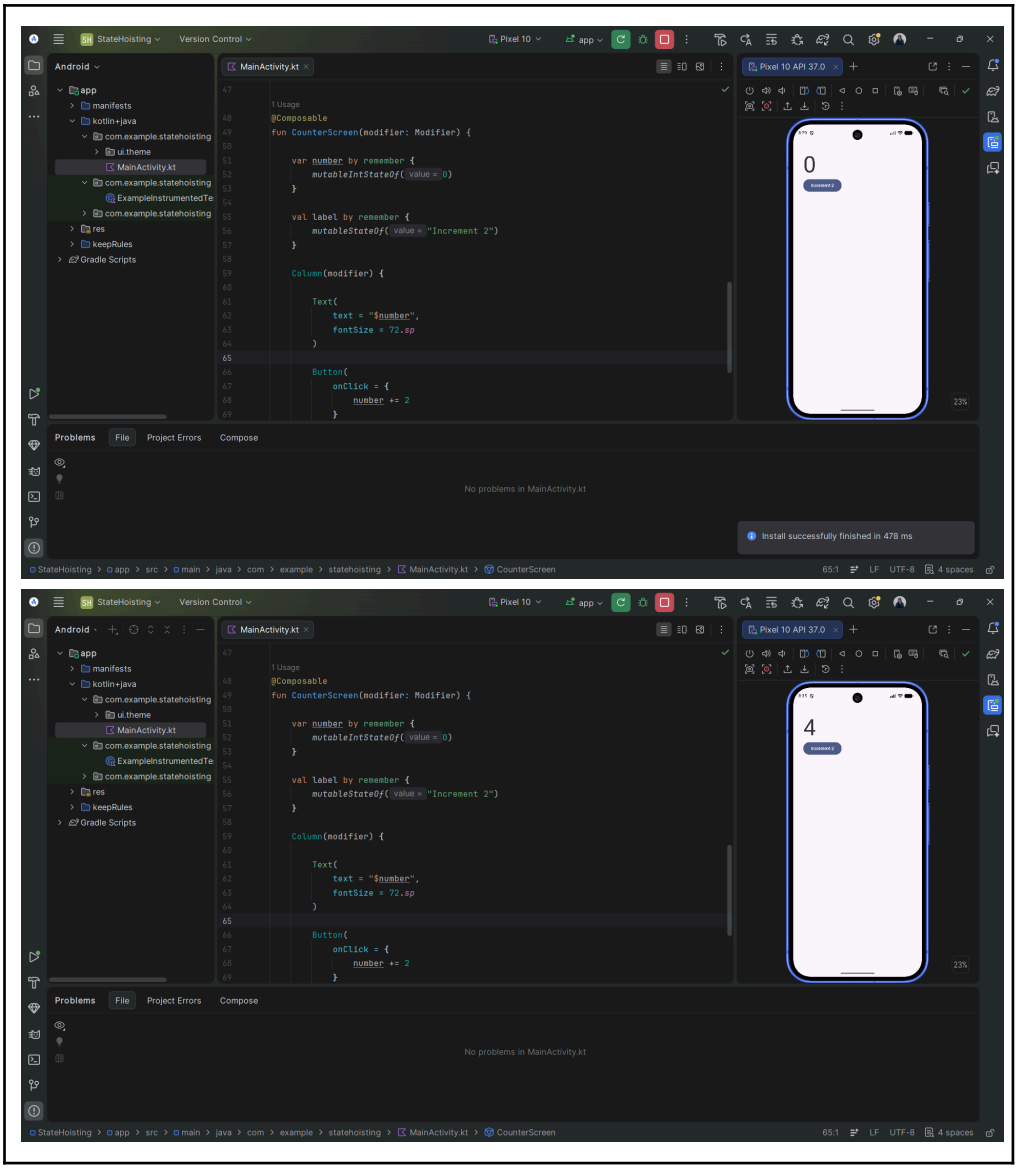
```
onClick = { number++ }
```

menjadi

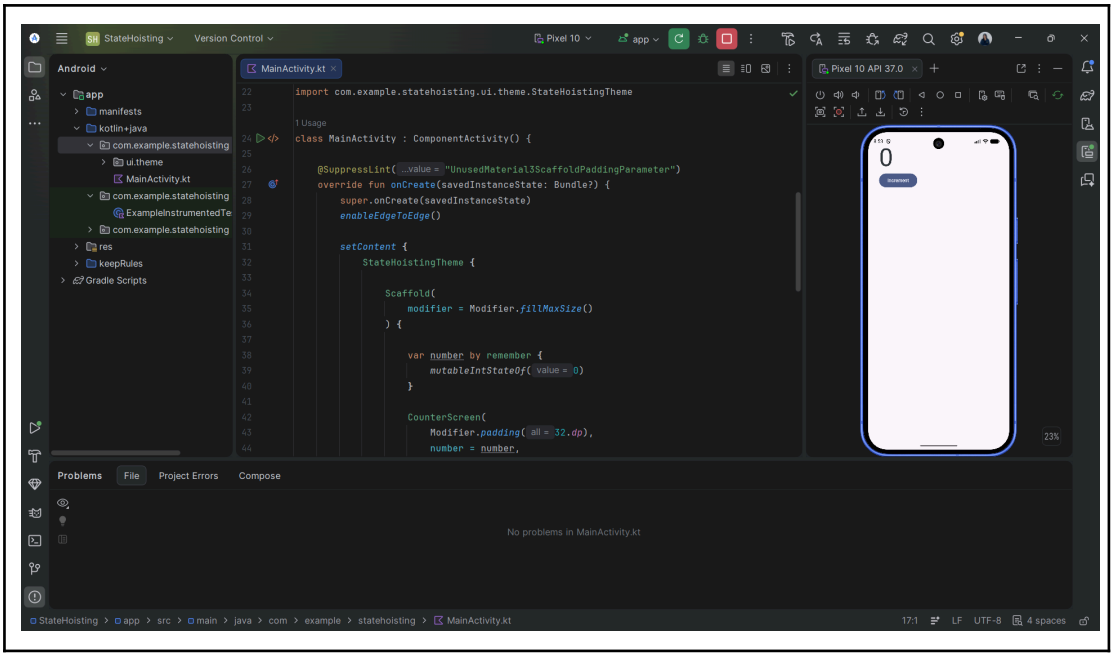
```
onClick = { number += 2 }
```

Dengan perubahan tersebut, nilai akan bertambah 2 setiap klik. Lalu untuk tombol akan menampilkan “Incement 2”

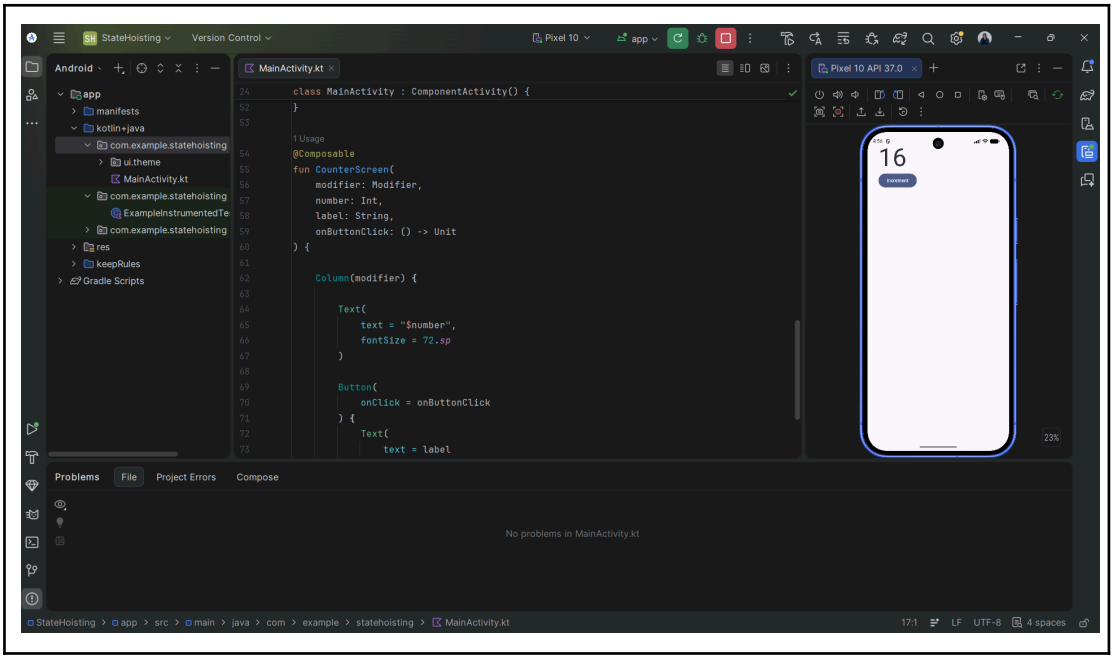
c. Screenshot sebelum dan sesudah



3. Melakukan state dan event hoisting



4. Setelah klik beberapa kali tombol Increment



b. Ya, program tetap berjalan dengan baik dan secara tampilan tetap menunjukkan perilaku yang sama, yaitu angka bertambah setiap kali tombol Increment diklik. Perbedaannya, setelah dilakukan state hoisting, pengelolaan state dan event tidak lagi berada di dalam CounterScreen, tetapi dipindahkan ke parent yang memanggil CounterScreen

c. Proses increment sekarang dilakukan pada callback onButtonClick ketika CounterScreen dipanggil

```
onButtonClick = { number++ }
```

Sedangkan di dalam CounterScreen, Button hanya menjalankan callback tersebut

```
Button(onClick = onButtonClick)
```

d. State number sekarang dideklarasikan di parent, sebelum CounterScreen dipanggil

```
var number by remember { mutableStateOf(0) }
```

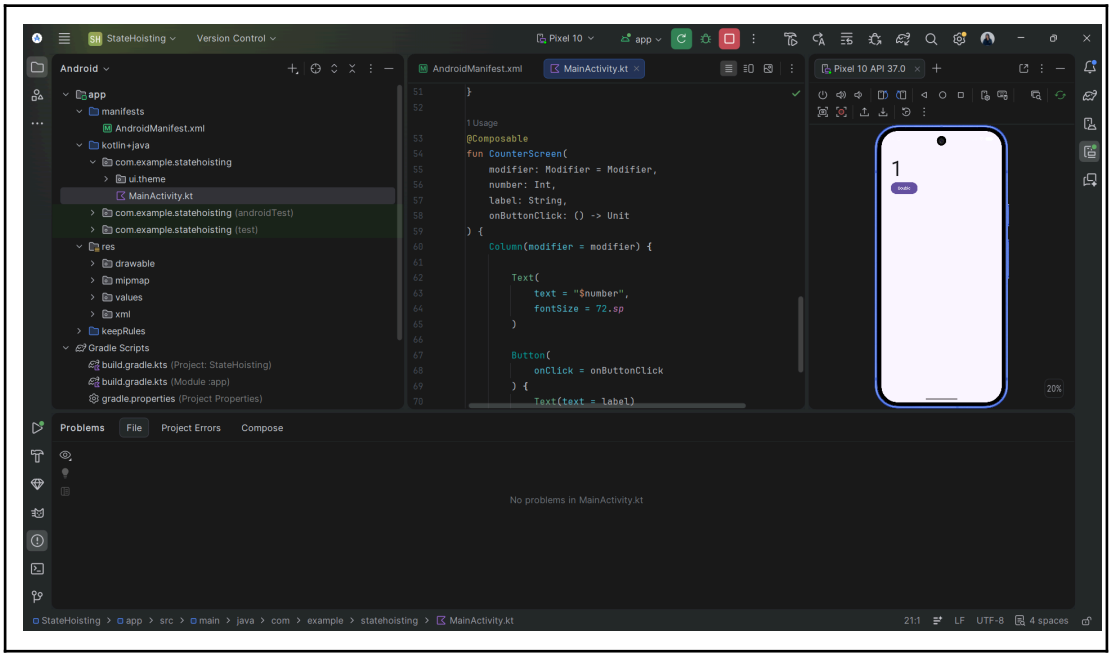
Nilai untuk label Button tidak lagi disimpan sebagai state di dalam CounterScreen, tetapi diberikan melalui parameter label = “Increment”

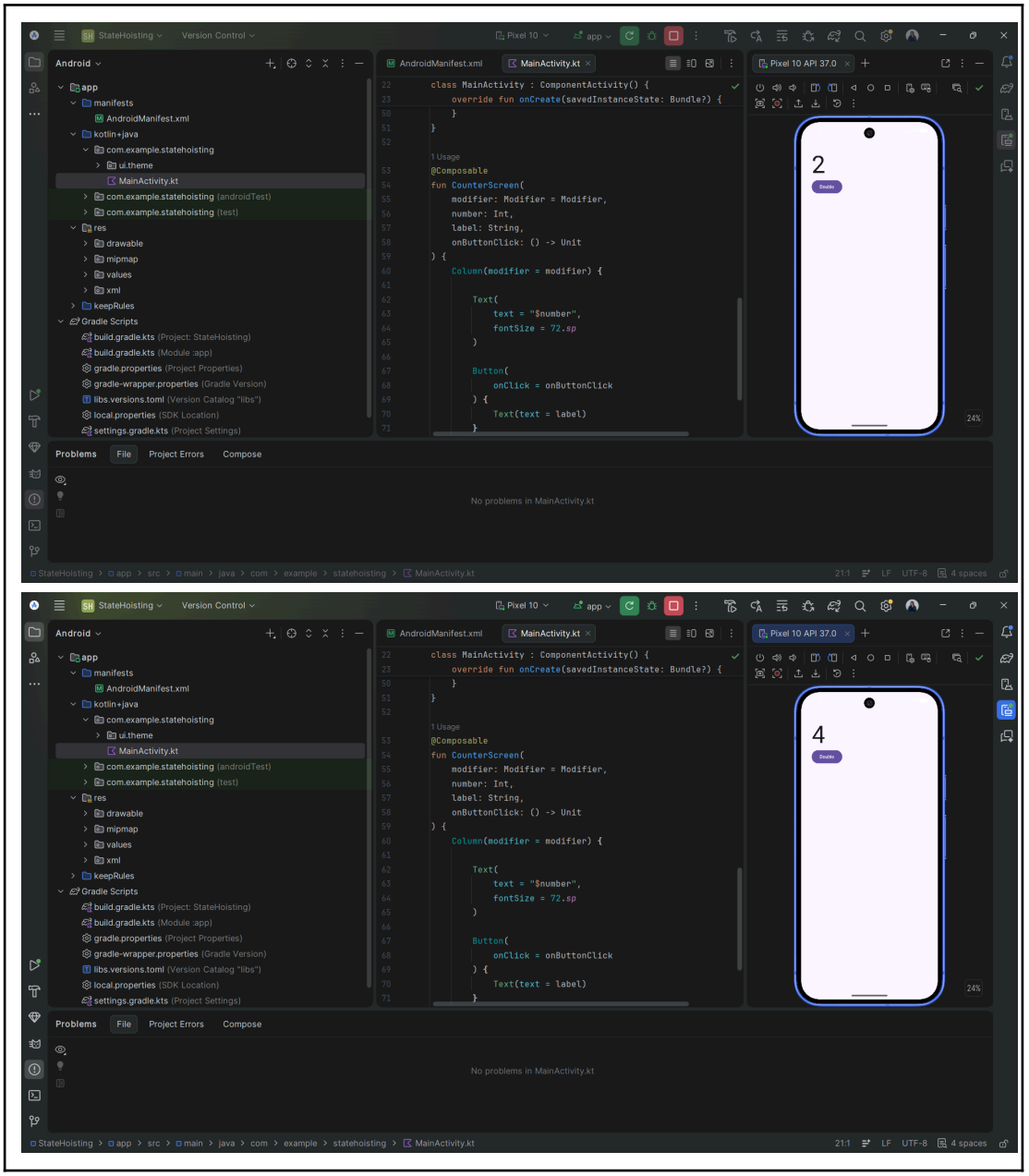
Kemudian keduanya diterima oleh CounterScreen melalui parameter:

```
number: Int,
label: String
```

Jadi, setelah state hoisting, CounterScreen hanya menerima data dan event dari parent, sementara state utama berada di luar CounterScreen

5. Ketika tombol ditekan, angka dikalikan 2, bukan ditambah 1.





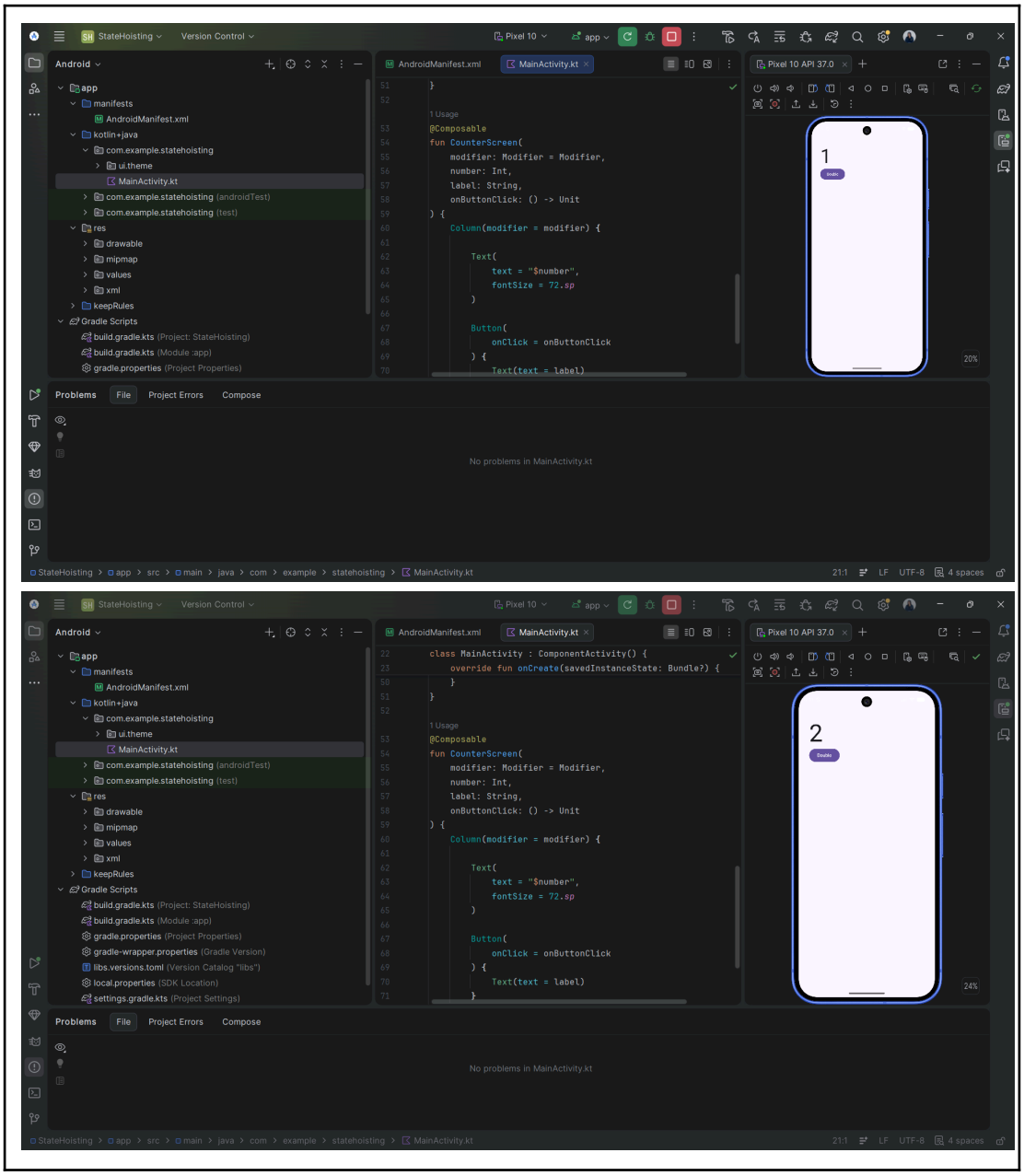
b. State number dideklarasikan di parent dengan nilai awal 1. Label tombol diubah menjadi "Double" dan fungsi onButtonClick menggunakan number *= 2 untuk menggandakan nilai setiap kali tombol diklik. Semua perubahan logika dilakukan di parent tanpa perlu mengubah fungsi CounterScreen.

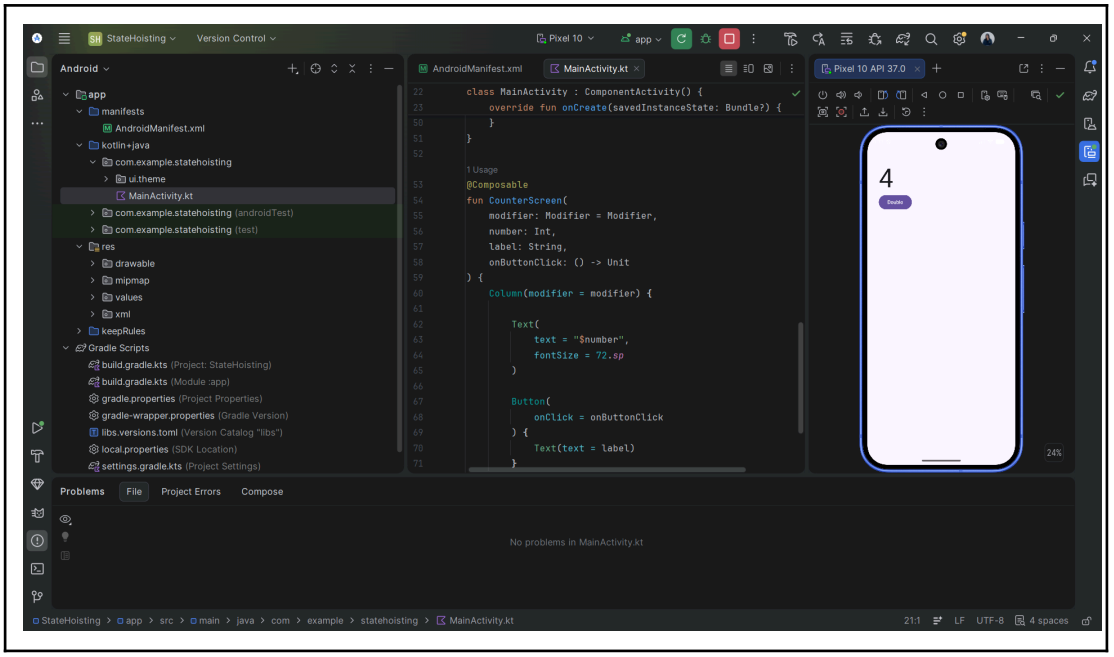
```
var number by remember {
    mutableStateOf(1)
}

CounterScreen(
    modifier = Modifier.padding(32.dp),
    number = number,
    label = "Double",
```

```
        onButtonClick = {
            number *= 2
        }
)
```

6. Screenshot sebelum dan sesudah nilai variabel number digandakan.





b. Perubahan kode dilakukan di dalam fungsi onCreate() pada class MainActivity, tepatnya pada deklarasi variabel state dan pemanggilan composable CounterScreen. Perubahan yang dilakukan adalah mengubah nilai awal number dari 0 menjadi 1, mengubah label tombol menjadi "Double", serta mengubah event onButtonClick dari number++ menjadi number *= 2.

```
var number by remember { mutableStateOf(0) }

CounterScreen(
    modifier = Modifier.padding(32.dp),
    number = number,
    label = "Increment",
    onButtonClick = { number++ }
)
```

```
var number by remember { mutableStateOf(1) }

CounterScreen(
    modifier = Modifier.padding(32.dp),
    number = number,
    label = "Double",
    onButtonClick = { number *= 2 }
)
```

c. Tidak ada perubahan yang dilakukan pada fungsi composable CounterScreen antara tahap state hoisting dan tahap penggandaan nilai. Hal ini karena CounterScreen hanya bertugas menampilkan nilai dan menjalankan callback yang diterima melalui parameter. Seluruh perubahan logika dilakukan pada parent, yaitu di dalam onCreate().

7. Penerapan state hoisting memberikan manfaat berupa kemudahan dalam mengubah kebutuhan fungsional aplikasi tanpa harus mengubah kode di dalam composable CounterScreen.

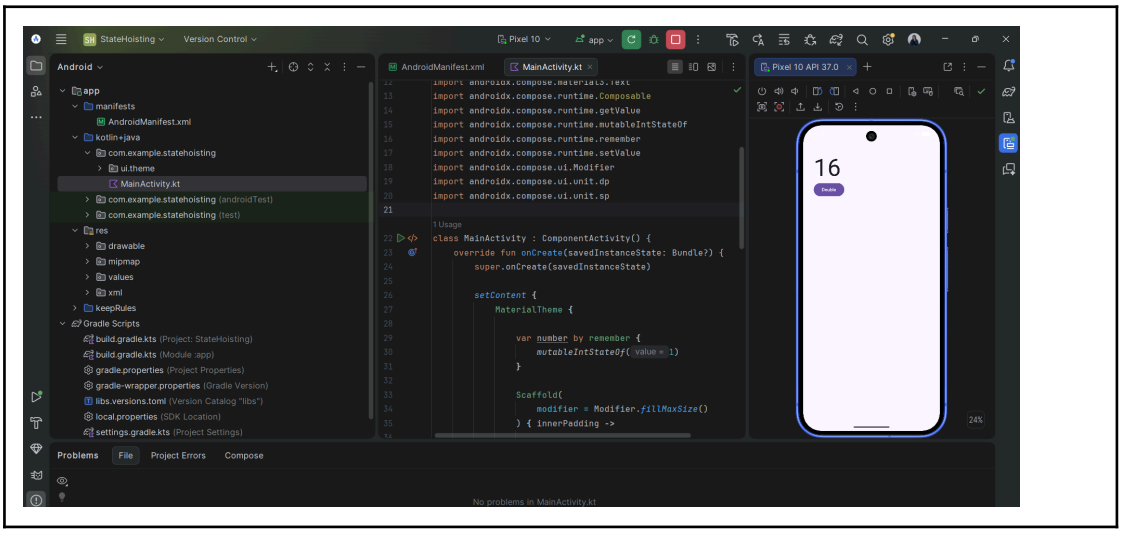
Hal ini terjadi karena pengelolaan state dan logika event telah dipindahkan ke parent, yaitu MainActivity. Sebagai contoh, perubahan tombol dari "Increment" menjadi "Double" dan logika dari penambahan menjadi perkalian cukup dilakukan pada pemanggilan CounterScreen.

Dengan demikian, penerapan state hoisting membuat kode lebih fleksibel, mudah dipelihara, dapat digunakan kembali (reusable), serta mendukung prinsip Single Source of Truth (SSoT).

Percobaan 2 (4.5.2 Perubahan Konfigurasi)

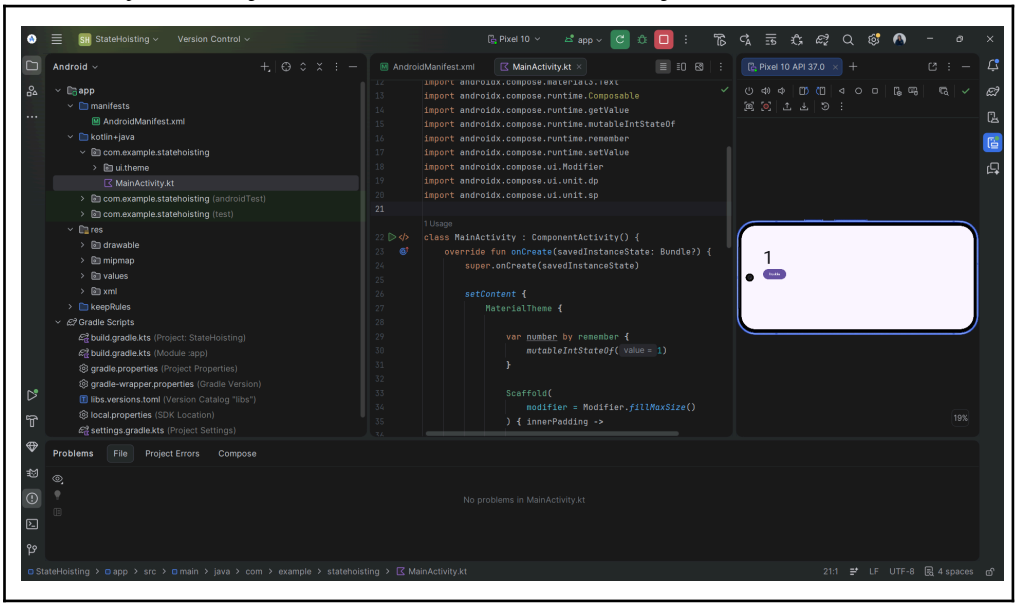
1. Setelah program dijalankan dalam posisi portrait, nilai awal variabel number adalah 1. Kemudian tombol "Double" diklik sebanyak empat kali sehingga nilai berubah menjadi 2, 4, 8, dan 16.

Hal ini terjadi karena setiap kali tombol diklik, event onButtonClick menjalankan operasi number *= 2 yang menggandakan nilai variabel number. Perubahan tersebut secara otomatis memperbarui tampilan melalui mekanisme recomposition pada Jetpack Compose.



2. Perubahan konfigurasi

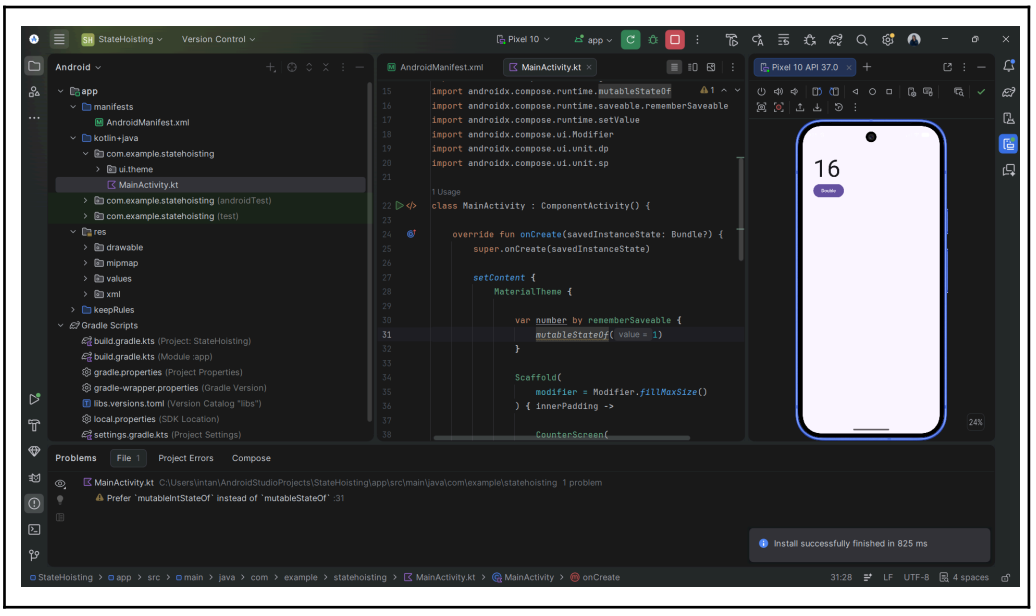
a. Setelah layar emulator diputar dari Portrait menjadi Landscape, angka yang sebelumnya menunjukkan 16 berubah kembali menjadi 1.

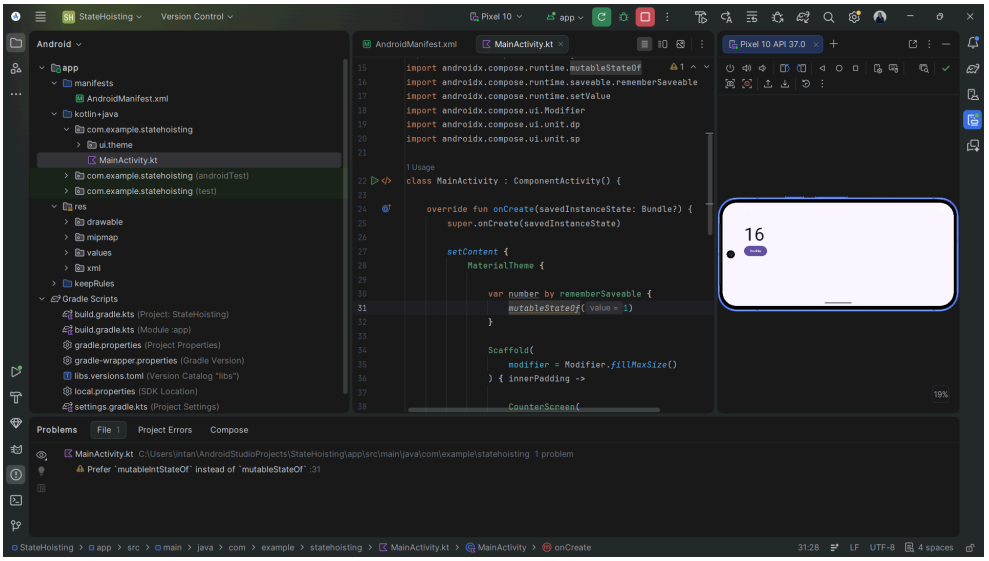


b. Angka yang ditampilkan adalah 1. Hal ini terjadi karena variabel number masih menggunakan remember { mutableStateOf(1) }. Ketika orientasi perangkat berubah, Activity secara default dibuat ulang sehingga state yang disimpan menggunakan remember tidak dapat dipertahankan. Akibatnya, variabel number kembali ke nilai awal yaitu 1.

3. Setelah perubahan konfigurasi

a. Setelah tombol Double diklik sebanyak 4 kali, angka yang ditampilkan adalah 16. Ketika layar diputar dari Portrait ke Landscape, angka tetap menunjukkan 16.





b. Angka yang ditampilkan tetap 16 karena variabel number sudah menggunakan rememberSaveable. Konstruk ini mempertahankan nilai state meskipun terjadi perubahan konfigurasi seperti rotasi layar, sehingga angka tidak kembali ke nilai awal 1.

c. rememberSaveable berfungsi untuk mempertahankan nilai state ketika terjadi perubahan konfigurasi, seperti rotasi layar dari Portrait ke Landscape. Berbeda dengan remember yang akan mengembalikan state ke nilai awal saat Activity dibuat ulang, rememberSaveable menyimpan state sehingga nilai yang sudah berubah tetap dipertahankan meskipun layar diputar.

4. Contoh penggunaan rememberSaveable adalah pada formulir pendaftaran pengguna yang memiliki input nama, email, dan nomor telepon. Dengan menggunakan rememberSaveable, data yang sudah diketik pengguna tetap tersimpan meskipun layar perangkat diputar dari Portrait ke Landscape.
Manfaatnya adalah mencegah hilangnya data input saat terjadi perubahan konfigurasi, sehingga pengguna tidak perlu mengetik ulang dan aplikasi menjadi lebih nyaman digunakan.

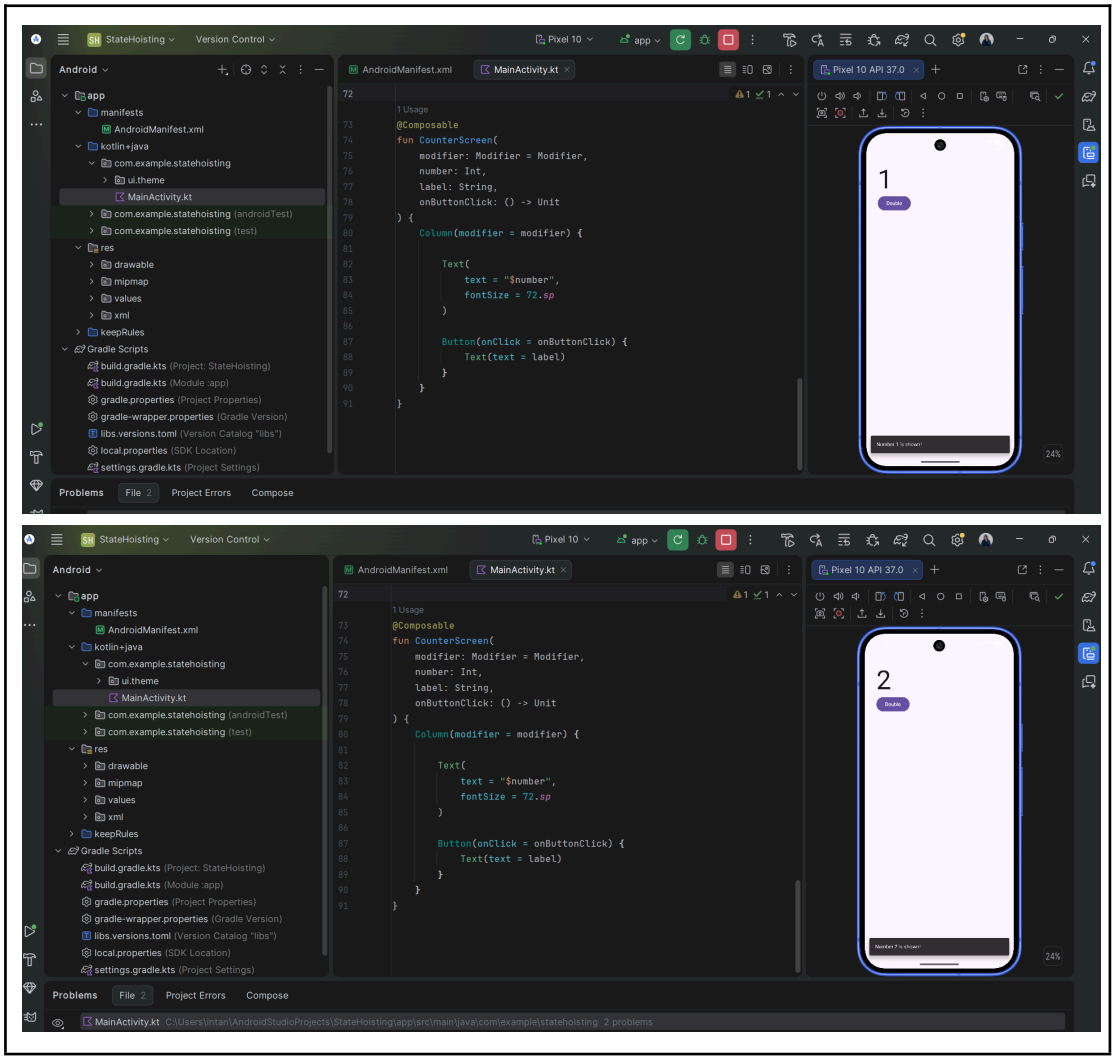
Contoh penerapan pada kode sebagai berikut.

```
@Composable
fun FormPendaftaran() {
    var nama by rememberSaveable {
        mutableStateOf("")
    }

    TextField(
        value = nama,
        onValueChange = { nama = it },
        label = { Text("Masukkan Nama") }
    )
}
```

Percobaan 3 (4.5.3 LaunchedEffect)

a. Percobaan 1



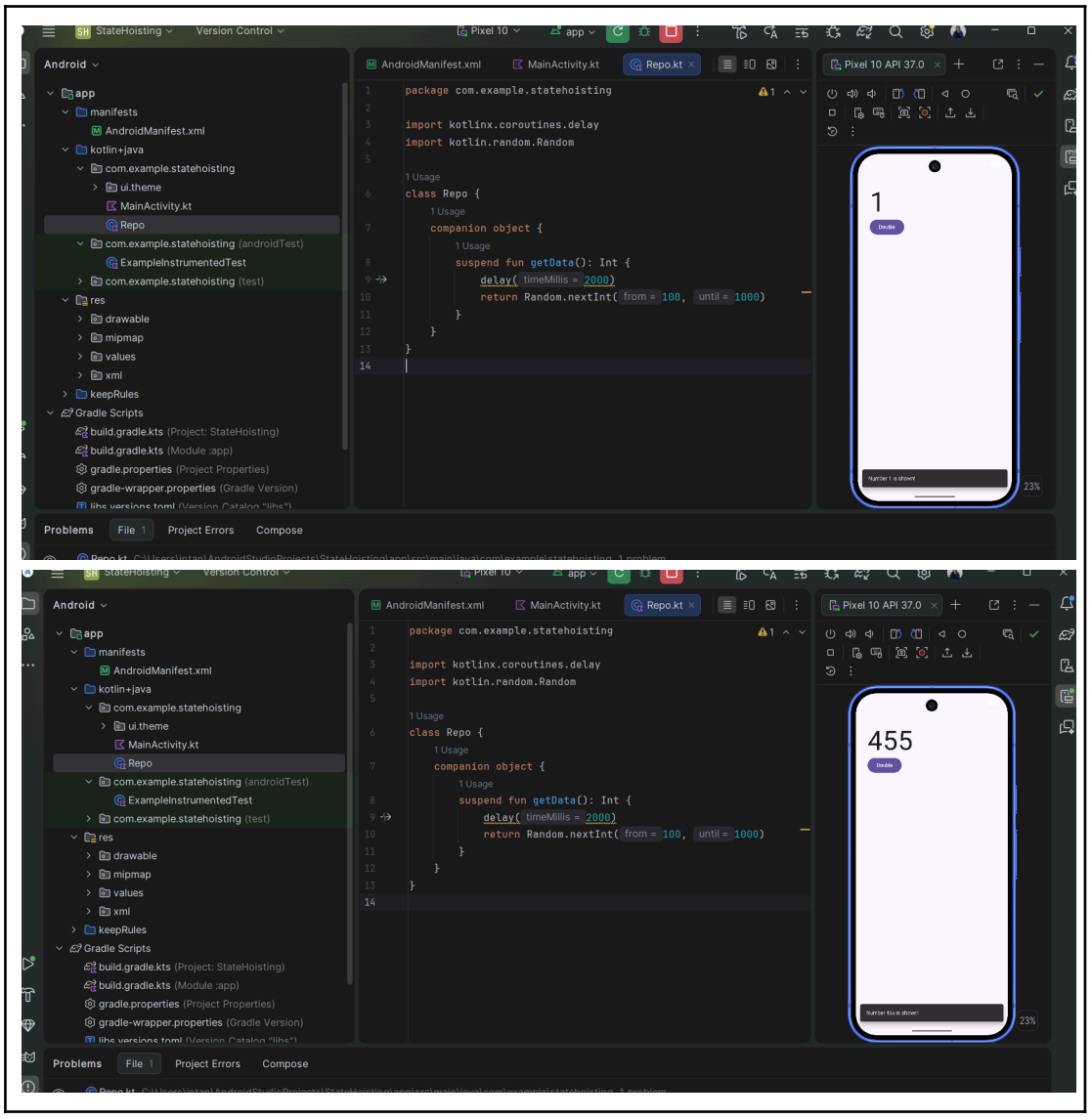
1. Setelah menjalankan program.
    a. Saat pertama kali dijalankan, aplikasi menampilkan angka 1, tombol Double, dan Snackbar dengan pesan "Number 1 is shown!" secara otomatis tanpa perlu menekan tombol.
    b. Pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa interaksi pengguna karena LaunchedEffect(number) otomatis dijalankan ketika composable pertama kali masuk ke komposisi, sehingga Snackbar muncul meskipun pengguna belum melakukan interaksi.
    c. Pesan dipicu oleh LaunchedEffect(number) yang berjalan saat pertama kali ditampilkan atau ketika nilai number berubah. Bagian kode yang menjalankannya adalah

```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar(
        "Number $number is shown!"
    )
```

```
}
```

2. Setelah klik Double klik satu kali
    a. Angka berubah dari 1 menjadi 2. Kemudian Snackbar menampilkan pesan "Number 2 is shown!" secara otomatis.
    b. Hal tersebut terjadi karena tombol Double menjalankan number *= 2 sehingga nilai state berubah dari 1 menjadi 2. Perubahan state ini memicu LaunchedEffect(number) untuk dijalankan kembali dan menampilkan Snackbar sesuai nilai terbaru.

b. Percobaan 2



1. Setelah menjalankan program
    a. Saat pertama kali dijalankan, aplikasi menampilkan angka 1, tombol Double, dan Snackbar dengan pesan "Number 1 is shown!".

b. Setelah dua detik, angka berubah secara otomatis menjadi angka acak antara 100 hingga 999. Snackbar juga menampilkan pesan sesuai angka baru tersebut.
c. Perubahan angka dipicu oleh LaunchedEffect(Unit) yang memanggil Repo.getData() setelah delay 2 detik. Perubahan state number kemudian memicu LaunchedEffect(number) untuk menampilkan Snackbar dengan nilai terbaru.
    - Angka yang mengubah angka secara otomatis
      MainActivity.kt

```
LaunchedEffect(Unit) {
    number = Repo.getData()
}
```

      Repo.kt

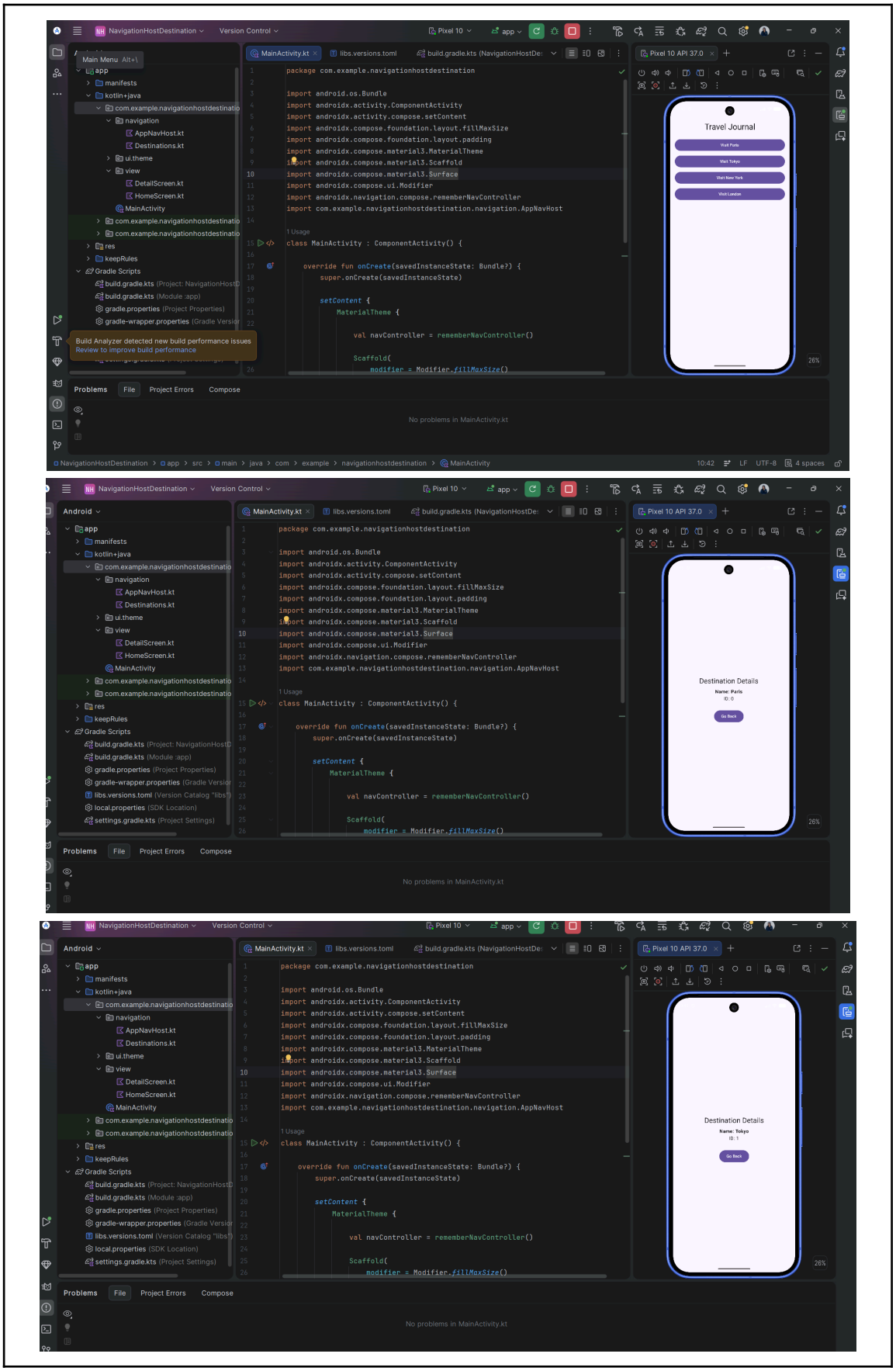
```
suspend fun getData(): Int {
    delay(2000)
    return Random.nextInt(100, 1000)
}
```

    - Kode yang menampilkan Snacbar
      MainActivity.kt

```
LaunchedEffect(number) {
    snackbarHostState.showSnackbar(
        "Number $number is shown!"
    )
}
```

2. Dari kedua percobaan,
    a. LaunchedEffect(state) akan dijalankan kembali setiap kali nilai state yang digunakan sebagai key berubah. Sedangkan LaunchedEffect(Unit) hanya dijalankan sekali ketika composable pertama kali masuk ke komposisi, selama composable tersebut tetap berada dalam komposisi yang sama.
    b. Operasi pada LaunchedEffect(state) dijalankan ketika composable pertama kali ditampilkan dan setiap kali nilai state berubah. Sementara itu, operasi pada LaunchedEffect(Unit) dijalankan saat composable pertama kali masuk ke komposisi dan tidak dijalankan ulang akibat perubahan state biasa.

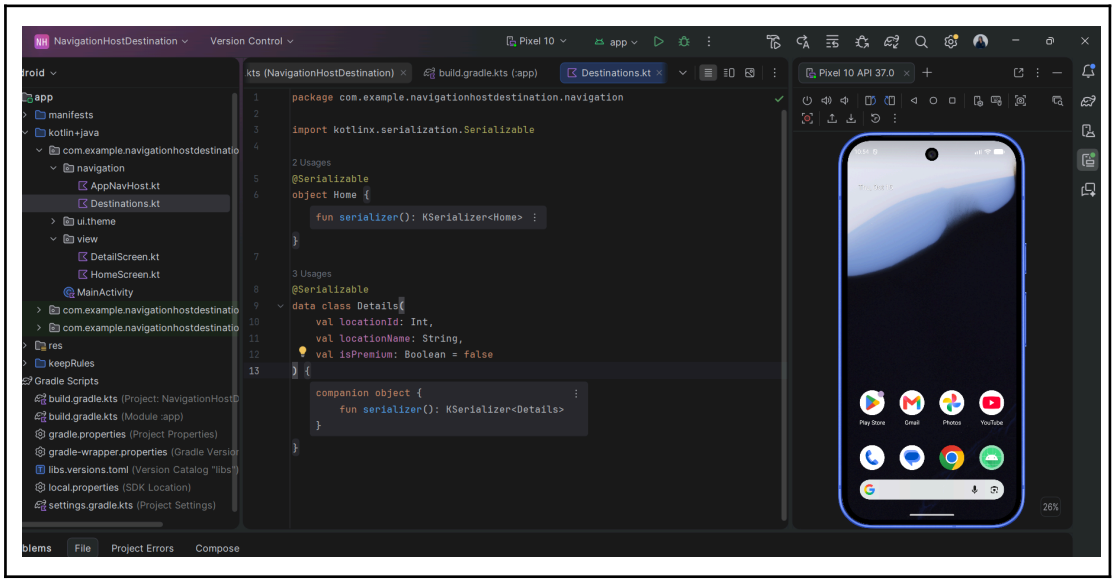
Percobaan 4 (6.5.1 Navigation Host dan Destination)



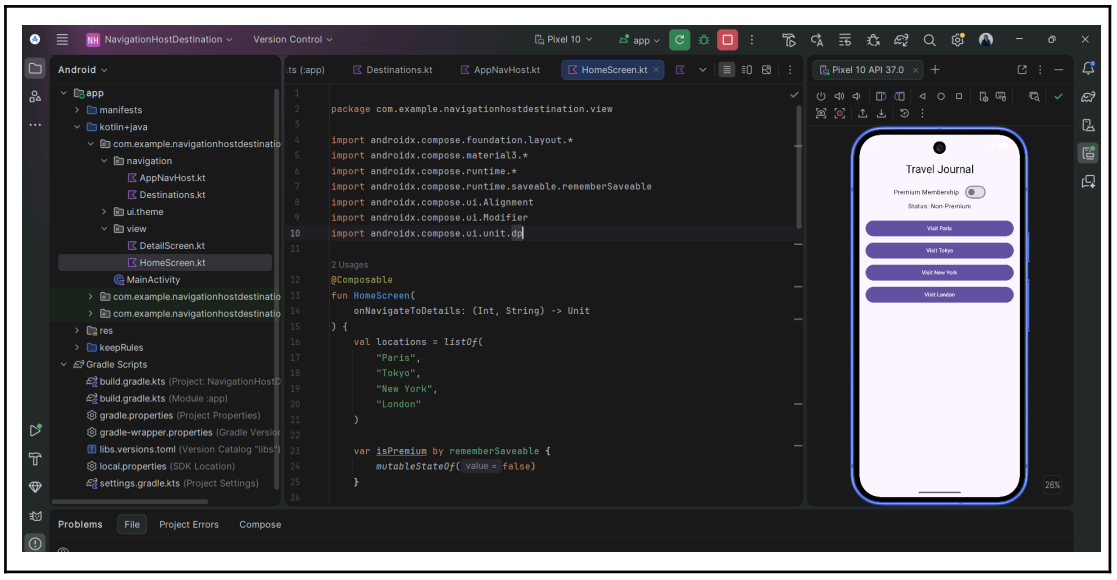
1. Mengubah class destination Details

```
@Serializable
data class Details(
    val locationId: Int,
    val locationName: String,
    val isPremium: Boolean = false
)
```

Class Details ditambahkan atribut isPremium bertipe Boolean dengan nilai default false. Atribut ini digunakan untuk menyimpan status premium tujuan wisata yang nantinya dikirim dari halaman HomeScreen ke DetailScreen melalui Navigation.



2. Menambahkan komponen Switch untuk mengubah nilai status membership dari false menjadi trus dan sebaliknya.



Kode yang ditambahkan adalah sebagai berikut.

```
var isPremium by rememberSaveable {
    mutableStateOf(false)
}

Switch(
    checked = isPremium,
    onCheckedChange = {
        isPremium = it
    }
)
```

Komponen Switch digunakan untuk mengubah status membership melalui variabel isPremium bertipe Boolean. Ketika Switch diaktifkan, nilai berubah menjadi true, sedangkan ketika dinonaktifkan nilainya menjadi false. Perubahan state ini secara otomatis diperbarui pada tampilan menggunakan Jetpack Compose.

3. Bagian kode yang diubah sebagai berikut.
    a. HomeScreen.kt

```
onNavigateToDetails: (Int, String, Boolean) -> Unit

onNavigateToDetails(index, name, isPremium)
```

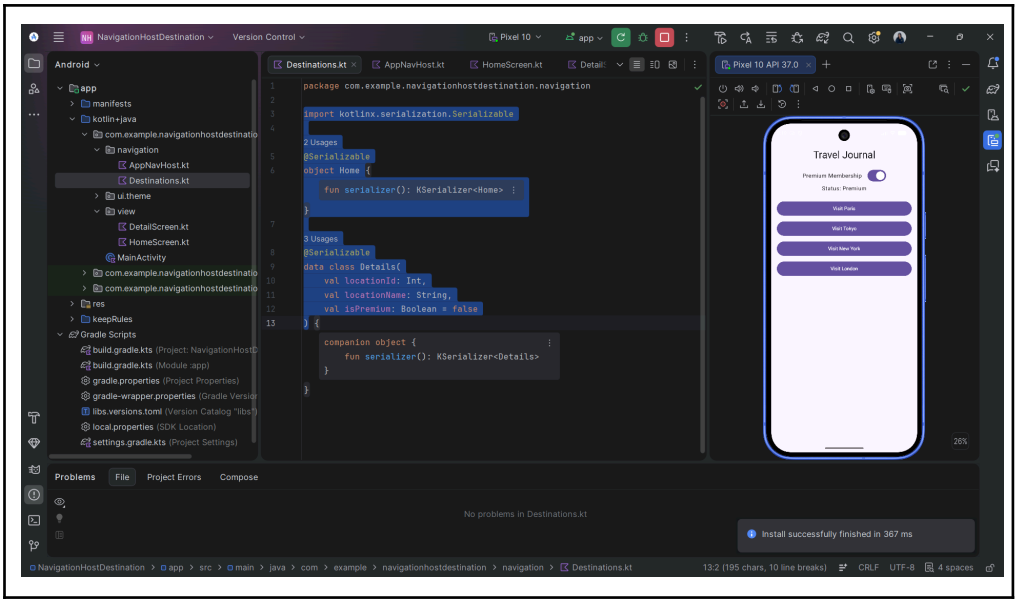
    b. AppNavHost.kt

```
navController.navigate(
    Details(
        locationId = id,
        locationName = name,
        isPremium = isPremium
    )
)
```

    c. DetailSCreen.kt

```
Text(
    text = if (isPremium) {
        "Status: Premium"
    } else {
        "Status: Non-Premium"
    }
)
```

Perubahan dilakukan pada HomeScreen.kt, AppNavHost.kt, dan DetailScreen.kt. Pada HomeScreen, parameter callback ditambahkan tipe Boolean untuk mengirim nilai isPremium. Kemudian AppNavHost meneruskan nilai tersebut melalui objek Details menggunakan Navigation. Selanjutnya, DetailScreen menerima dan menampilkan status Premium sesuai nilai yang dikirim. Perubahan ini diperlukan agar status membership dari halaman Home dapat ditampilkan pada halaman Detail.



4. Mengubah warna menjadi gold/emas pada fitur premium.
   a. Penambahan kode pada DetailScreen.kt

   Menambahkan kondisi warna background dengan kode berikut.

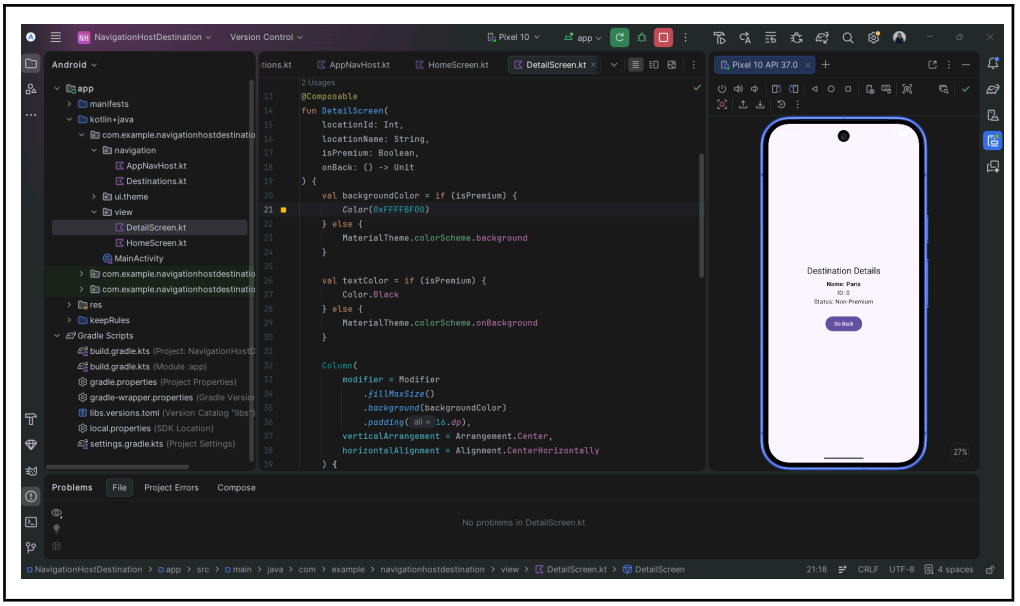
```
val backgroundColor = if (isPremium) {
    Color(0xFFFFD700)
} else {
    MaterialTheme.colorScheme.background
}
```

   Lalu menerapkan watna background pada column dengan kode berikut.

```
modifier = Modifier
    .fillMaxSize()
    .background(backgroundColor)
    .padding(16.dp)
```

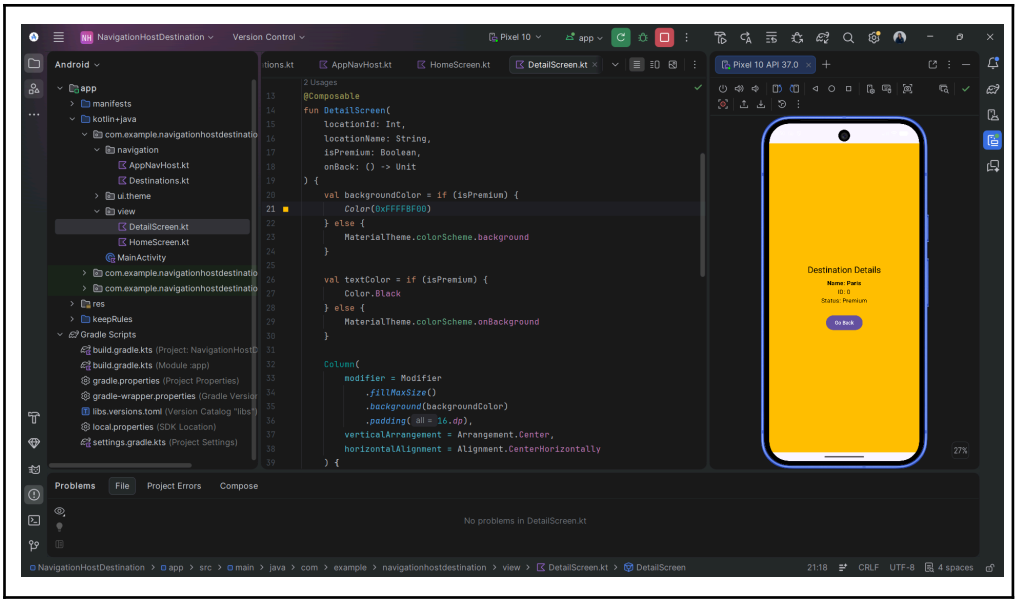
   Perubahan dilakukan pada file DetailScreen.kt dengan menambahkan kondisi if (isPremium) untuk menentukan warna background. Jika isPremium bernilai true, background berubah menjadi warna Gold (0xFFFFD700), sedangkan jika bernilai false, background menggunakan warna default dari tema aplikasi.

   b. Screenshot Switch OFF



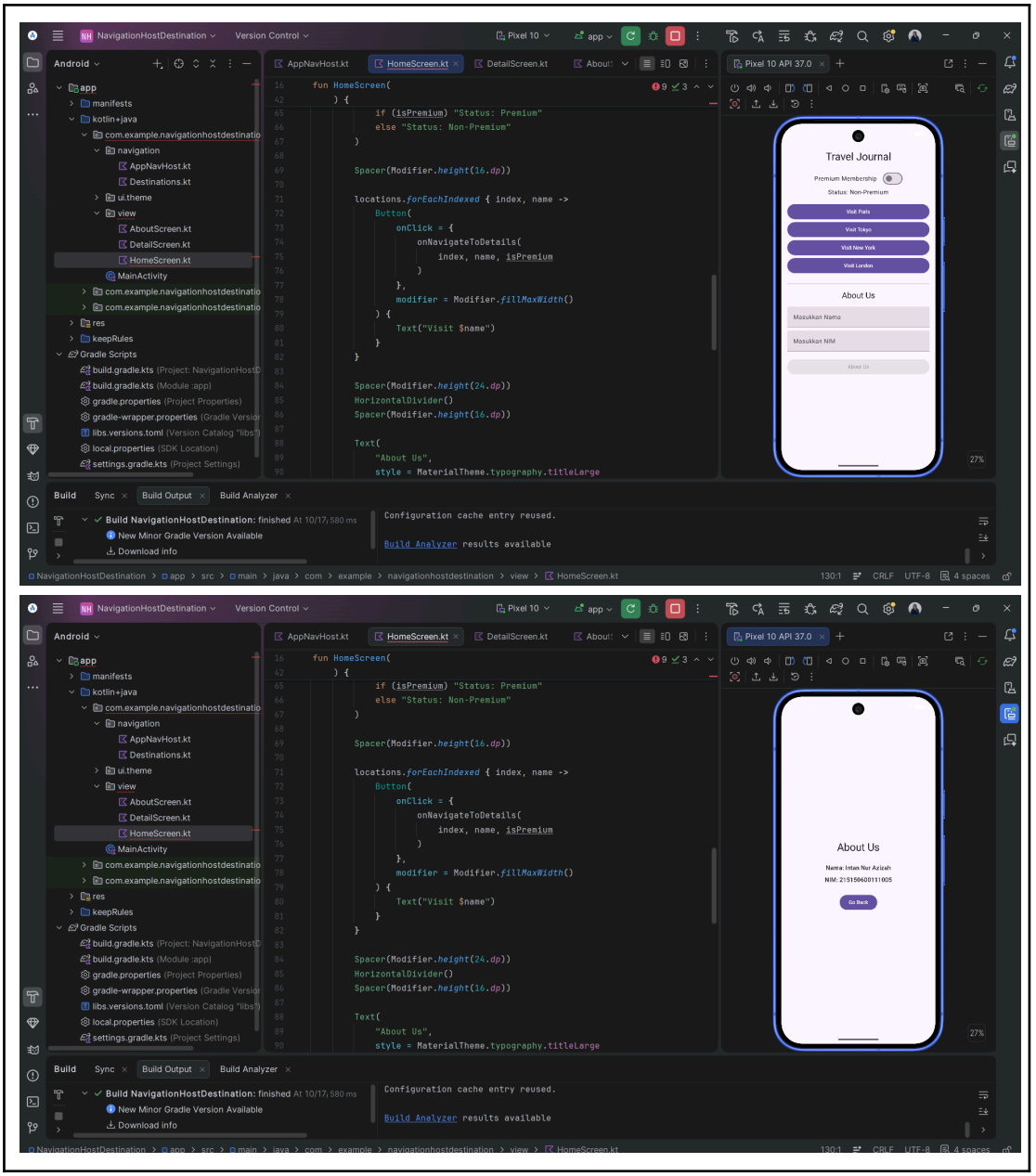
Ketika Switch Premium dalam posisi OFF, nilai `isPremium` adalah `false`. Halaman DetailScreen menampilkan status Non-Premium dengan warna background default.

c. Screenshot Switch ON



Ketika Switch Premium dalam posisi ON, nilai isPremium berubah menjadi true. Halaman DetailScreen menampilkan status Premium dengan warna background Gold (Emas).

5. Membuat halaman About dengan berisikan nama dan NIM berdasarkan kustomisasi atau input.



a. Untuk membuat halaman About, ditambahkan composable AboutScreen yang menampilkan nama dan NIM. Kemudian, pada AppNavHost.kt ditambahkan rute berbasis String "about" agar halaman dapat diakses menggunakan navController.navigate("about"). Dengan demikian, pengguna dapat berpindah dari HomeScreen menuju halaman About.
Kode pada AboutScreen.kt

```
@Composable
fun AboutScreen(
    name: String,
    nim: String,
    onBack: () -> Unit
) {
    Column {
```

```
        Text("About Us")
        Text("Nama: $name")
        Text("NIM: $nim")

        Button(onClick = onBack) {
            Text("Go Back")
        }
    }
}
```

Kode pada AppNavHost.kt

```
composable("about") {
    AboutScreen(
        name = "Nama Mahasiswa",
        nim = "NIM Mahasiswa",
        onBack = {
            navController.popBackStack()
        }
    )
}
```

Kode untuk membuka halaman sebagai berikut.

```
navController.navigate("about")
```

b. Agar nama bersifat dinamis, ditambahkan TextField pada HomeScreen untuk menerima input pengguna. Pada implementasi ini, NIM juga dibuat dinamis. Kedua nilai dikirim melalui route "about/{name}/{nim}" menggunakan navController.navigate(), kemudian diterima melalui backStackEntry.arguments dan ditampilkan pada AboutScreen. Perubahan ini diperlukan agar nama dan NIM dapat ditampilkan sesuai input pengguna tanpa harus mengubah kode program secara manual.
Kode pada HomeScreen.kt

```
var nama by rememberSaveable {
    mutableStateOf("")
}

var nim by rememberSaveable {
    mutableStateOf("")
}

TextField(
    value = nama,
    onValueChange = { nama = it },
```

```
    label = { Text("Masukkan Nama") }
)

TextField(
    value = nim,
    onValueChange = { nim = it },
    label = { Text("Masukkan NIM") }
)
```

Kode navigasi pada AppNavHost.kt

```
navController.navigate(
    "about/${Uri.encode(nama)}/${Uri.encode(nim)}"
)
```

Kode penerima data pada AppNavHost.kt.

```
composable("about/{name}/{nim}") { backStackEntry
->

    val name = backStackEntry.arguments
        ?.getString("name") ?: ""

    val nim = backStackEntry.arguments
        ?.getString("nim") ?: ""

    AboutScreen(
        name = name,
        nim = nim,
        onBack = {
            navController.popBackStack()
        }
    )
}
```

6. Navigasi Type-Safe sebaiknya digunakan ketika aplikasi memiliki banyak halaman dan membutuhkan pengiriman data dengan tipe tertentu, seperti Int, String, atau Boolean, karena lebih aman dan dapat mendeteksi kesalahan tipe data saat kompilasi. Contohnya adalah navigasi dari HomeScreen ke DetailScreen yang mengirimkan ID lokasi, nama lokasi, dan status Premium. Sementara itu, navigasi berbasis String lebih cocok digunakan untuk aplikasi sederhana dengan rute yang mudah dikelola, seperti navigasi menuju halaman About, Settings, atau Help. Pendekatan String lebih mudah diterapkan, tetapi lebih rentan terhadap kesalahan penulisan rute.

   Contoh kode Type-Safe Navigation

```
navController.navigate(
    Details(
        locationId = 1,
        locationName = "Tokyo",
        isPremium = true
    )
)
```

Contoh kode String-Based Navigation

```
navController.navigate("about")
```

Type-Safe lebih tepat untuk navigasi kompleks yang membutuhkan keamanan tipe data, sedangkan String-Based cocok untuk navigasi sederhana dengan rute yang tidak terlalu kompleks.

Percobaan 5 (6.5.2 Komunikasi Data Antar *Composable* Destination)

1. Memperbarui file Destinations.kt dengan menambahkan kode

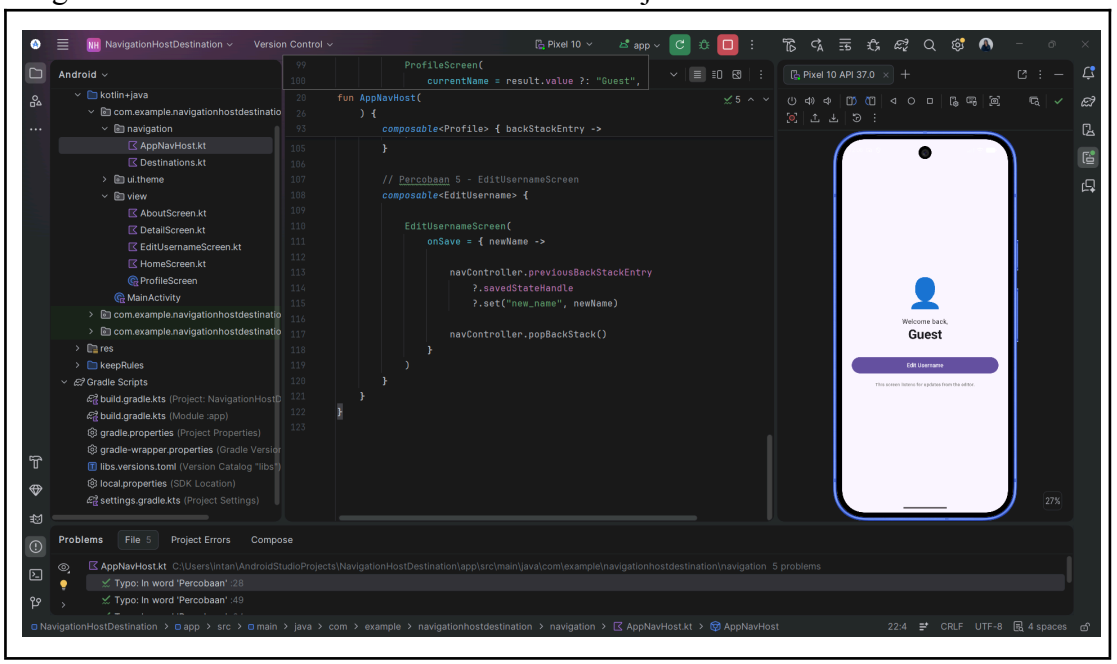
```
@Serializable
object Profile

@Serializable
object EditUsername
```

   a. Penambahan objek Profile dan EditUsername bertujuan untuk mendefinisikan dua destination baru dalam sistem navigasi Jetpack Compose. Objek Profile digunakan sebagai tujuan halaman profil, sedangkan EditUsername digunakan sebagai tujuan halaman untuk mengubah username. Kedua objek tersebut memungkinkan perpindahan halaman menggunakan pendekatan Type-Safe Navigation.
   b. Anotasi @Serializable digunakan agar objek Kotlin dapat diserialisasi oleh Kotlin Serialization. Dalam Navigation Compose, anotasi ini memungkinkan objek Profile dan EditUsername digunakan sebagai rute navigasi berbasis tipe (Type-Safe Navigation), sehingga navigasi lebih aman dan mengurangi risiko kesalahan penulisan rute.
   c. Jika anotasi @Serializable dihilangkan, objek Profile dan EditUsername tidak memiliki serializer yang diperlukan oleh Type-Safe Navigation. Akibatnya, ketika objek tersebut digunakan sebagai destination, aplikasi dapat mengalami error karena sistem navigasi tidak dapat melakukan serialisasi rute. Oleh karena itu, anotasi @Serializable diperlukan agar navigasi berbasis objek tersebut
2. Setelah menambahkan composable EditUsernameScreen dan ProfileScreen, aplikasi belum dapat menampilkan kedua halaman tersebut melalui navigasi. Hal ini karena

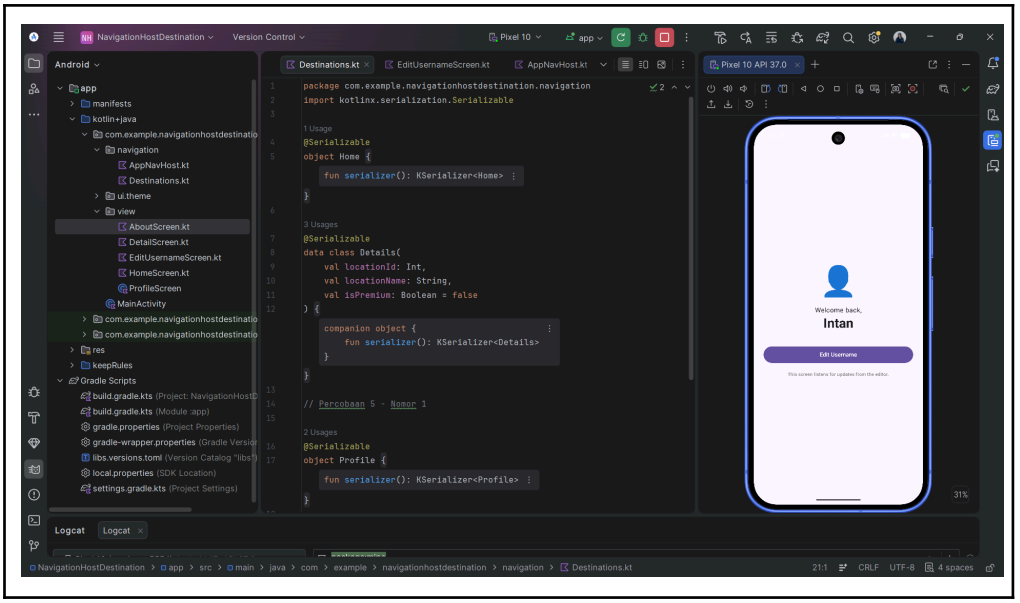
kedua composable belum didaftarkan sebagai destination pada AppNavHost. Oleh karena itu, perlu ditambahkan rute Profile dan EditUsername ke dalam NavHost agar kedua halaman dapat diakses dan saling berkomunikasi.

3. Perubahan kode dilakukan pada AppNavHost.kt dengan menambahkan destination Profile dan EditUsername. Pada ProfileScreen, SavedStateHandle digunakan untuk mengamati perubahan username melalui getLiveData() dan observeAsState(). Sementara itu, EditUsernameScreen mengirim nama baru melalui previousBackStackEntry.savedStateHandle.set(), kemudian menjalankan popBackStack() untuk kembali ke halaman Profile. Cara ini membuat komunikasi antarhalaman lebih terstruktur, mempertahankan data saat perubahan konfigurasi, dan menghindari penumpukan halaman pada back stack.
4. Pengubahan nilai startDestination dari Home menjadi Profile.

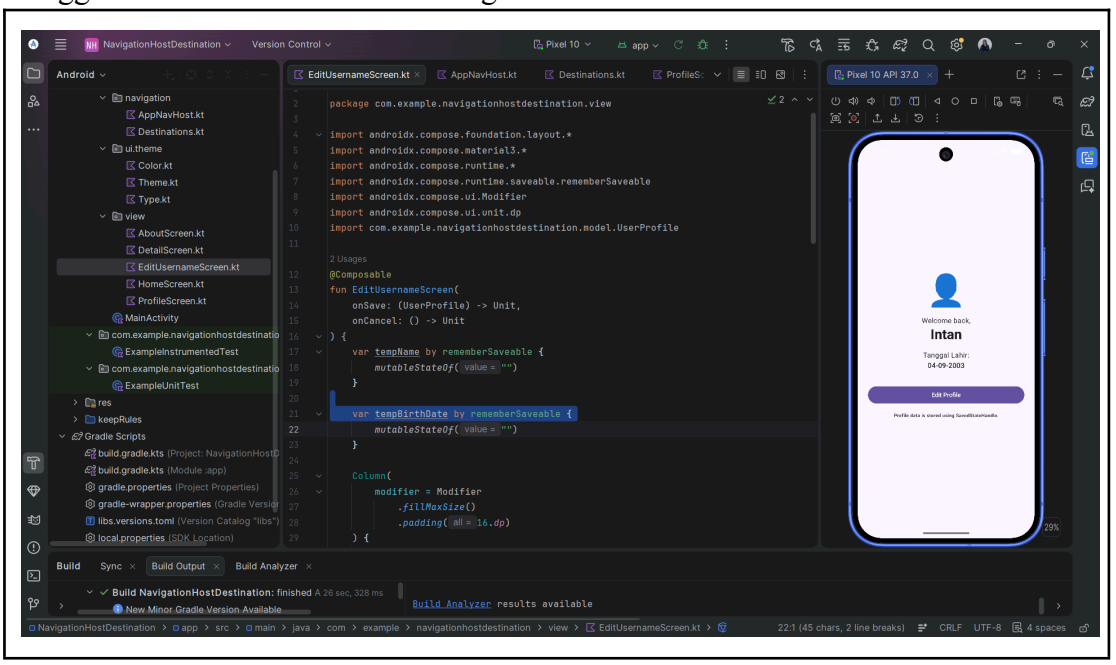


Saat pertama kali program dijalankan, halaman yang ditampilkan adalah ProfileScreen dengan username awal "Guest". Hal ini terjadi karena parameter startDestination pada NavHost telah diubah dari Home menjadi Profile, sehingga sistem navigasi menjadikan halaman Profile sebagai halaman pertama yang ditampilkan ketika aplikasi dijalankan.

5. Penambahan Button “Cancel” ke dalam composable EditUsernameScreen.
    a. Ya, ProfileScreen tetap mempertahankan nama sebelumnya karena tombol Cancel hanya menjalankan navController.popBackStack() tanpa menyimpan username baru ke dalam SavedStateHandle. Oleh karena itu, nama yang sebelumnya tersimpan tidak mengalami perubahan.
    b. Sebelum tombol Cancel diklik, ProfileScreen menampilkan username "Intan" yang telah disimpan sebelumnya. Setelah memasukkan nama "Azizah" pada EditUsernameScreen dan menekan Cancel, aplikasi kembali ke ProfileScreen dengan username tetap "Intan". Hal ini membuktikan bahwa tombol Cancel membatalkan perubahan tanpa memperbarui data pengguna.



6. Pada percobaan ini, dilakukan perubahan tipe data yang dikirim antarhalaman dari String menjadi custom data class UserProfile yang memiliki dua atribut, yaitu username dan birthDate. Class tersebut menggunakan anotasi @Parcelize dan mengimplementasikan Parcelable agar objek dapat disimpan menggunakan SavedStateHandle. Pada EditUsernameScreen, kedua nilai input digabungkan menjadi objek UserProfile dan dikirim ke ProfileScreen melalui previousBackStackEntry.savedStateHandle.set(). ProfileScreen kemudian menerima objek tersebut menggunakan getLiveData<UserProfile>() dan observeAsState(), sehingga username dan tanggal lahir dapat ditampilkan secara bersamaan. Penerapan ini memungkinkan pengiriman data kompleks antar halaman tanpa perlu menggunakan database atau variabel global.



7. Penjelasan

a. Mekanisme return-result memungkinkan pengguna berpindah ke halaman lain untuk mengubah atau memilih data, kemudian kembali ke halaman sebelumnya dengan membawa hasil perubahan tersebut. Sedangkan navigasi forward-only hanya memindahkan pengguna ke halaman berikutnya secara berurutan. Return-result lebih fleksibel untuk proses pengeditan data karena pengguna dapat kembali tanpa membuat halaman sebelumnya secara berulang.
b. Mekanisme return-result sebaiknya digunakan ketika pengguna perlu memilih atau mengubah data pada halaman lain, kemudian mengirimkan hasilnya kembali ke halaman sebelumnya. Contohnya adalah mengedit username, memilih kontak, menentukan tanggal lahir, atau memilih foto profil. Data yang diperbarui dapat dikirim kembali menggunakan SavedStateHandle. Contoh kode sebagai berikut.

```
navController.previousBackStackEntry
    ?.savedStateHandle
    ?.set("user_data", updatedProfile)

navController.popBackStack()
```

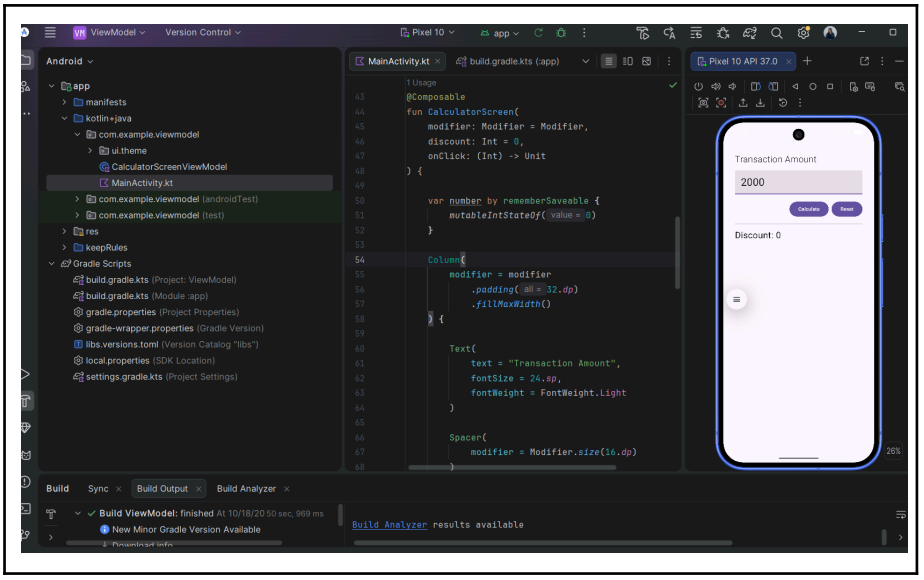
c. Navigasi forward-only sebaiknya digunakan ketika aplikasi memiliki alur proses yang berurutan dari satu halaman ke halaman berikutnya. Contohnya adalah proses registrasi akun, onboarding pengguna baru, atau checkout pembelian yang terdiri dari beberapa tahap. Mekanisme ini cocok karena pengguna mengikuti tahapan secara sistematis hingga proses selesai.
Contoh kode sebagai berikut.

```
navController.navigate("halaman_berikutnya")
```

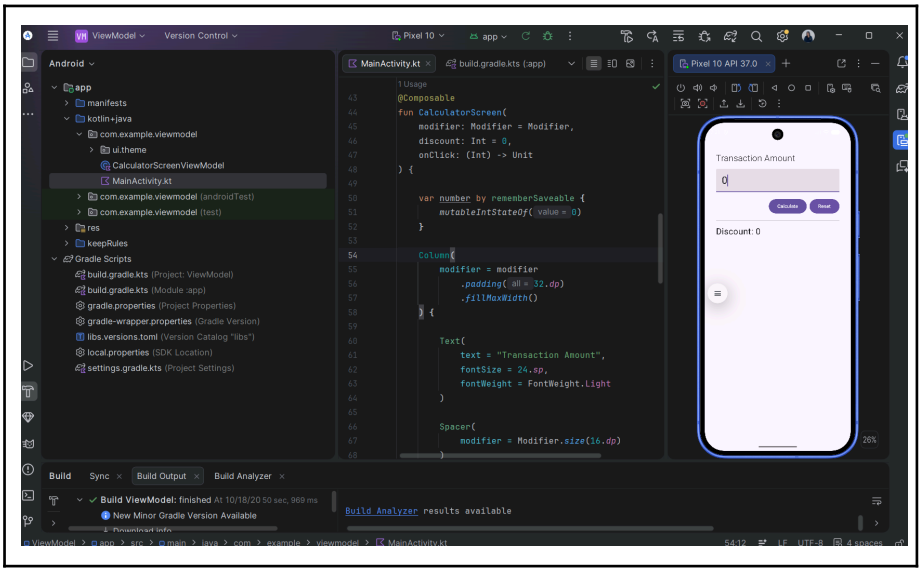
Percobaan 6 (7.5.1 ViewModel)

a. Percobaan 1

1. aa
    a. Setelah memasukkan nilai transaksi sebesar 20000 dan menekan tombol Calculate, nilai Discount tetap menampilkan 0. Hal ini terjadi karena tombol Calculate belum dihubungkan dengan fungsi compute() pada ViewModel, sehingga proses perhitungan diskon belum dijalankan.

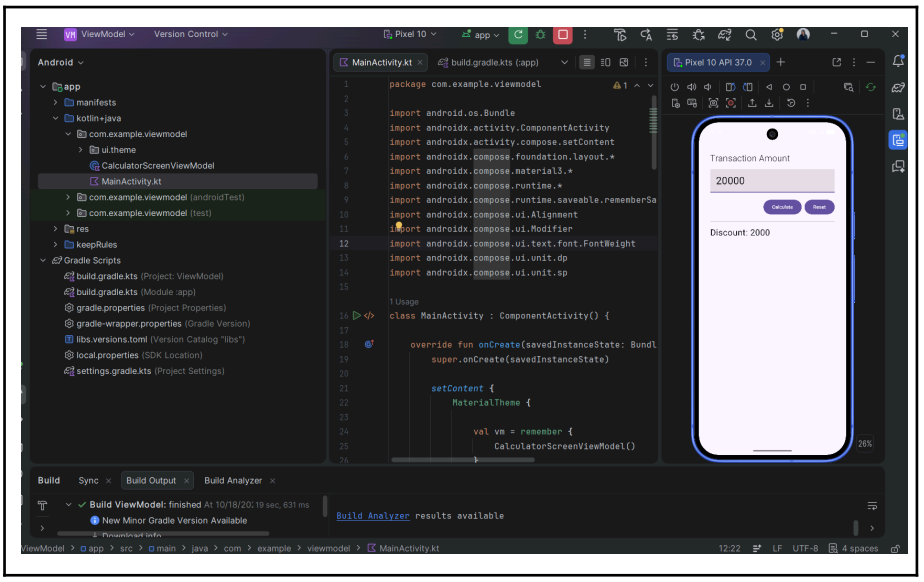


b. Setelah tombol Reset diklik, nilai pada TextField kembali menjadi 0. Hal ini terjadi karena event onClick pada tombol Reset menjalankan perintah number = 0, sehingga state number berubah dan tampilan diperbarui secara otomatis oleh Jetpack Compose.

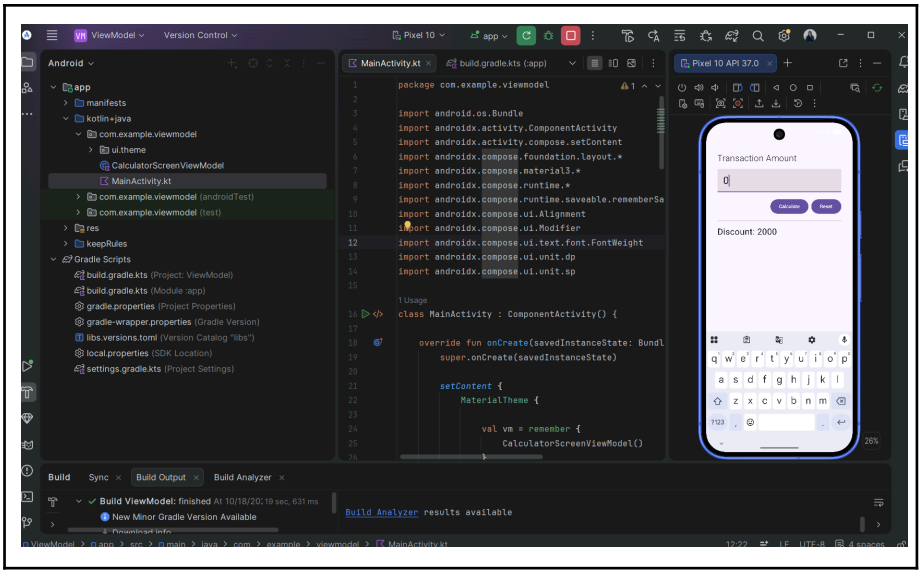


2. Pada tahap ini, dilakukan instansiasi objek CalculatorScreenViewModel dan pengambilan state discount menggunakan collectAsState(). Fungsi tersebut digunakan untuk mengamati perubahan nilai diskon dari StateFlow milik ViewModel agar nantinya dapat ditampilkan secara otomatis pada CalculatorScreen. Namun, nilai diskon belum terhubung ke tampilan.
3. Melakukan binding
    a. Setelah memasukkan nilai transaksi 20000 dan menekan tombol Calculate, nilai Discount berubah menjadi 2000. Hal ini terjadi karena event onClick memanggil fungsi vm.compute(number) pada ViewModel yang menghitung diskon sebesar 10%. Hasil perhitungan disimpan dalam MutableStateFlow, kemudian diamati menggunakan

collectAsState() sehingga tampilan Discount diperbarui secara otomatis.



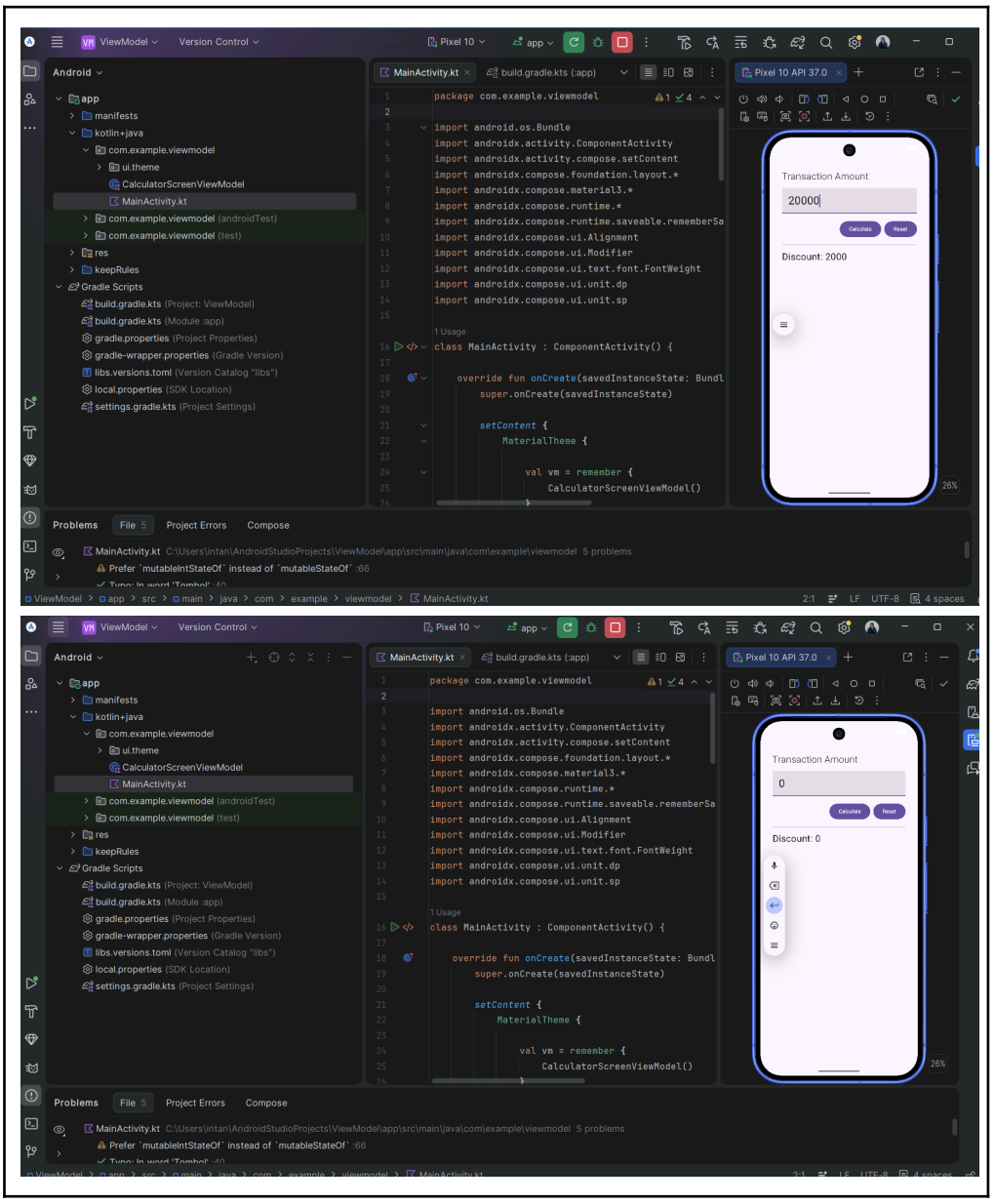
b. Ketika tombol Reset diklik, nilai Transaction Amount kembali menjadi 0, tetapi Discount tetap menunjukkan 2000. Hal ini terjadi karena tombol Reset hanya menjalankan number = 0 untuk mengubah state lokal pada CalculatorScreen dan tidak memanggil fungsi pada ViewModel untuk mengatur ulang nilai diskon.



c. Tombol Calculate menjalankan fungsi compute() pada ViewModel untuk menghitung dan memperbarui nilai diskon melalui StateFlow, sedangkan tombol Reset hanya mengubah state lokal number menjadi 0 pada CalculatorScreen. Oleh karena itu, Calculate memengaruhi hasil diskon, sementara Reset hanya mengosongkan nilai transaksi tanpa mengubah diskon yang tersimpan pada ViewModel.

4. Perubahan dilakukan dengan menambahkan fungsi reset() pada CalculatorScreenViewModel untuk mengubah nilai _discount menjadi 0.

Kemudian, pada CalculatorScreen ditambahkan parameter callback onReset yang dihubungkan dengan fungsi vm.reset() di MainActivity. Ketika tombol Reset diklik, nilai input number diubah menjadi 0 dan callback onReset() dijalankan untuk mengatur ulang diskon melalui ViewModel. Perubahan nilai diskon pada StateFlow kemudian diamati menggunakan collectAsState(), sehingga tampilan diperbarui secara otomatis. Penerapan ini memenuhi prinsip Single Source of Truth karena pengelolaan nilai diskon hanya dilakukan di dalam ViewModel.



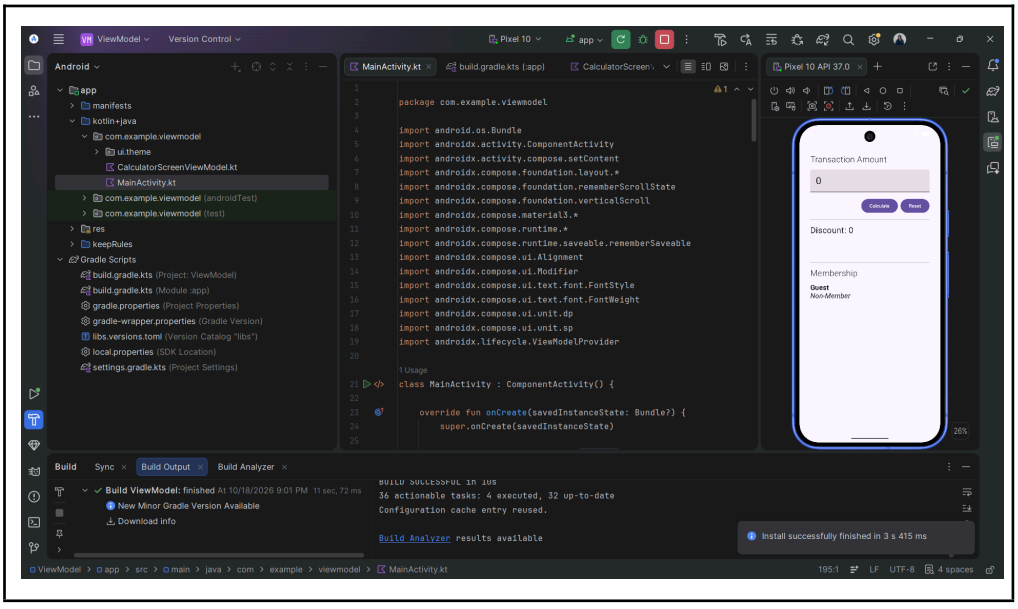

```
// CalculatorScreenViewModel.kt
fun reset() {
    _discount.update { 0 }
}
```

```
// MainActivity.kt
onReset = {
    vm.reset()
}

// CalculatorScreen
Button(
    onClick = {
        number = 0
        onReset()
    }
) {
    Text("Reset")
}
```

b. Percobaan 2

1.

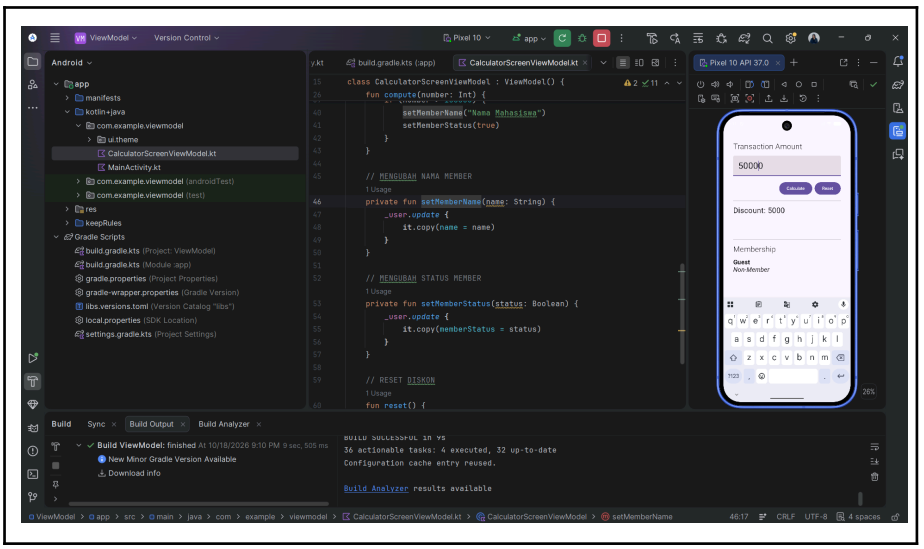


a. Perbedaan tampilan aplikasi pada Percobaan 2 adalah adanya penambahan bagian Membership di bawah kalkulator diskon. Pada Percobaan 1, aplikasi hanya menampilkan Transaction Amount, tombol Calculate dan Reset, serta hasil Discount. Sedangkan pada Percobaan 2, aplikasi juga menampilkan nama pengguna dan status membership yang diperoleh dari objek User pada ViewModel.

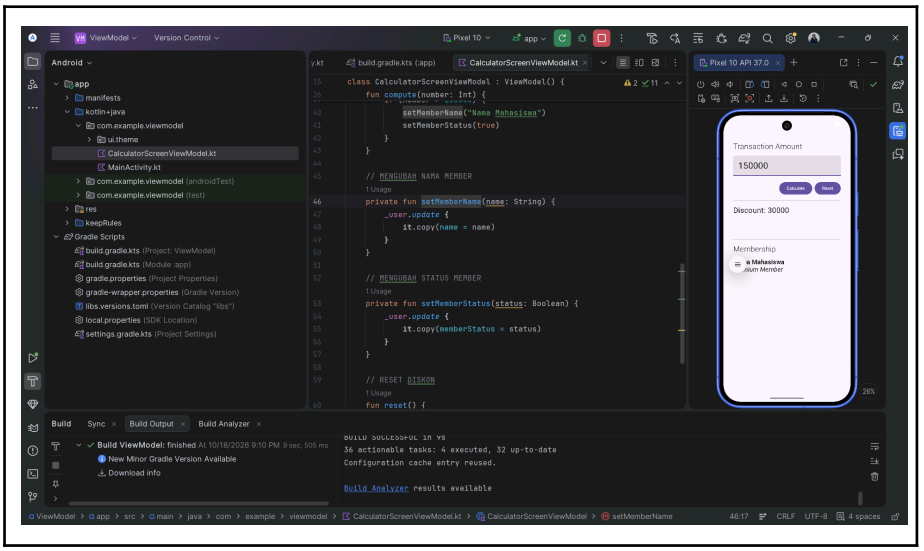
b. Informasi yang ditampilkan pada aplikasi terdiri dari nilai Transaction Amount, hasil perhitungan Discount, nama pengguna, dan status membership. Pada saat aplikasi pertama kali dijalankan, nilai Transaction Amount adalah 0, Discount sebesar 0, nama pengguna

adalah Guest, dan status membership adalah Non-Member karena nilai awal memberStatus adalah false

2. Penambahan logika diskon dan membership.
    a. Ketika nilai transaksi sebesar 50000 dimasukkan dan tombol Calculate diklik, aplikasi menghasilkan diskon sebesar 5000 atau 10% dari total transaksi. Nama pengguna tetap Guest dan status membership tetap Non-Member karena nilai transaksi tidak melebihi 100000.



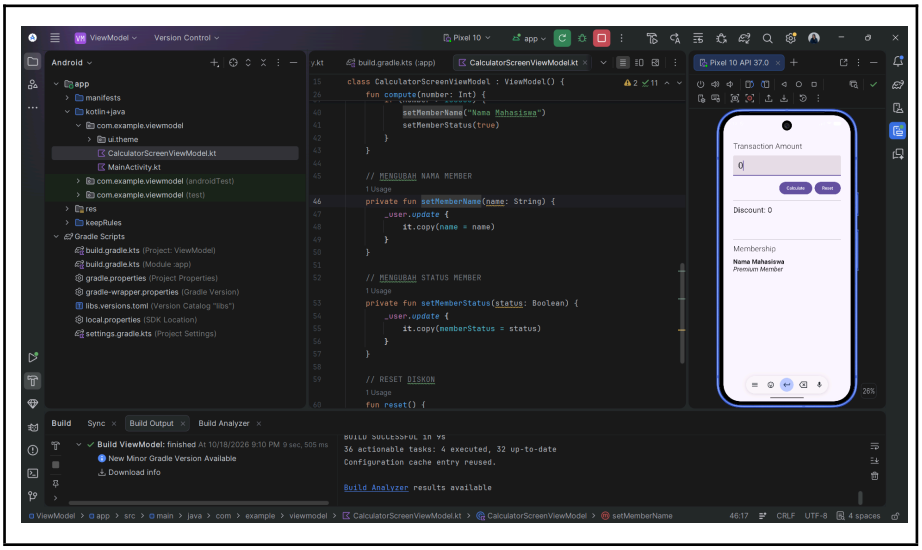
    b. Ketika nilai transaksi diubah menjadi 150000 dan tombol Calculate diklik, aplikasi menghasilkan diskon sebesar 30000 atau 20% dari total transaksi. Selain itu, nama pengguna berubah menjadi nama mahasiswa dan status membership berubah menjadi Premium Member karena nilai transaksi melebihi 100000.



    c. Perbedaan pada kedua pengujian terletak pada persentase diskon dan informasi membership. Pada transaksi 50000, aplikasi memberikan diskon 10% sebesar 5000 dengan status Guest dan Non-Member. Sedangkan pada transaksi 150000, aplikasi memberikan diskon 20%

sebesar 30000 serta mengubah nama pengguna menjadi nama mahasiswa dan status menjadi Premium Member. Perubahan tersebut terjadi karena fungsi compute() memiliki kondisi if (number > 100000).

3. Menekan tombol reset.
    a. Ketika tombol Reset diklik, nilai Transaction Amount kembali menjadi 0 dan nilai Discount juga berubah menjadi 0. Namun, nama pengguna dan status membership tidak mengalami perubahan. Nama pengguna tetap menampilkan nama mahasiswa dan statusnya tetap Premium Member.

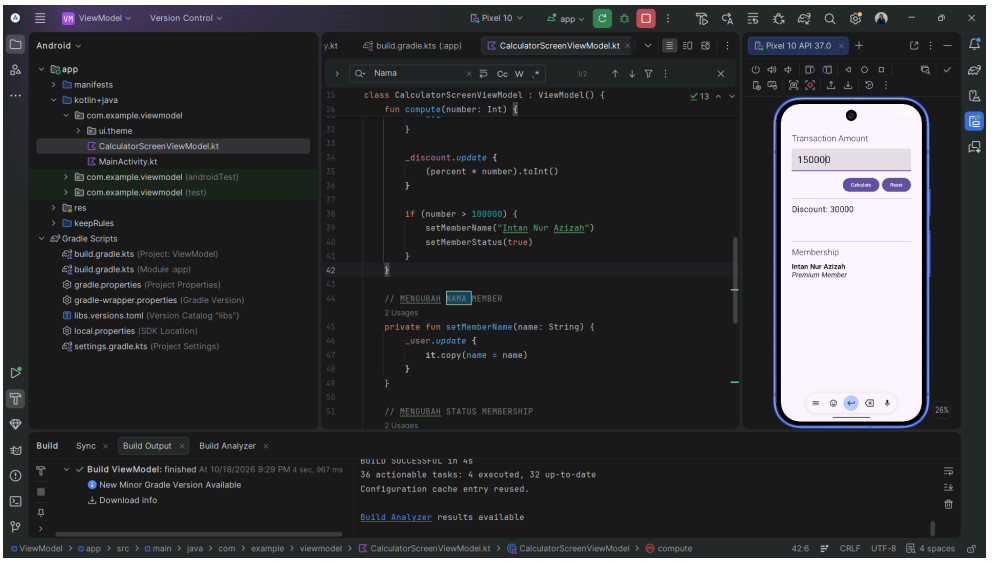


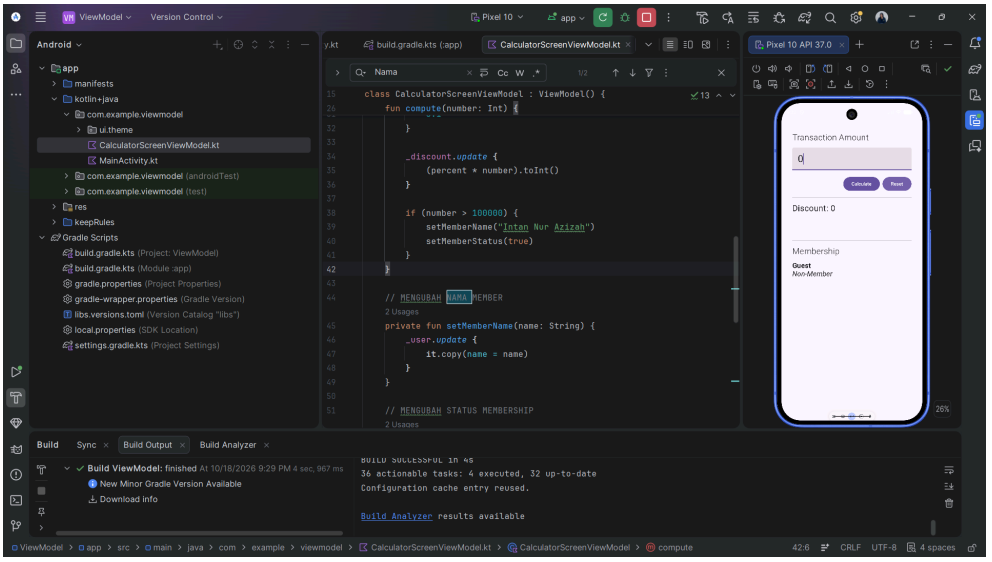
    b. Nama pengguna dan status membership tidak berubah karena fungsi reset() pada ViewModel hanya mengatur ulang nilai _discount menjadi 0. Fungsi tersebut belum memanggil setMemberName() dan setMemberStatus() untuk mengembalikan data pengguna ke kondisi awal. Oleh karena itu, objek User yang tersimpan dalam StateFlow tetap mempertahankan nama mahasiswa dan status Premium Member.
4. Perubahan kode dilakukan dengan menambahkan pemanggilan fungsi setMemberName("Guest") dan setMemberStatus(false) di dalam fungsi reset() pada CalculatorScreenViewModel. Ketika tombol Reset diklik, nilai Transaction Amount dikembalikan menjadi 0 dan fungsi vm.reset() dijalankan untuk mengatur ulang diskon, nama pengguna, serta status membership. Perubahan data dilakukan melalui MutableStateFlow di dalam ViewModel, kemudian diamati oleh UI menggunakan collectAsState(), sehingga tampilan otomatis diperbarui. Hasilnya, diskon kembali menjadi 0, nama pengguna berubah menjadi Guest, dan status membership berubah menjadi Non-Member.

```
fun reset() {
    _discount.update {
        0
```

```
        }

        setMemberName("Guest")
        setMemberStatus(false)
}
```





5. sdsa
    a. Penerapan prinsip Single Source of Truth (SSoT) pada nilai diskon dilakukan dengan menjadikan CalculatorScreenViewModel sebagai satu-satunya sumber data diskon. Nilai diskon disimpan dalam MutableStateFlow melalui variabel _discount dan hanya dapat diubah melalui fungsi compute() serta reset() pada ViewModel. Sementara itu, CalculatorScreen hanya mengamati perubahan nilainya menggunakan collectAsState(). Dengan demikian, data diskon dikelola secara

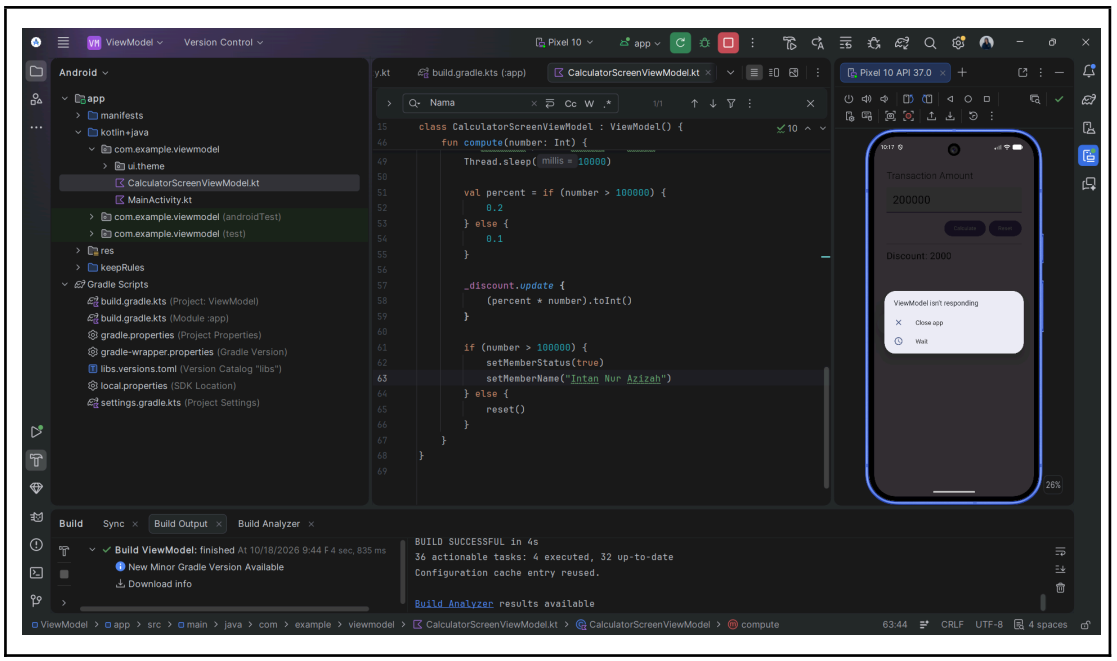
terpusat dan tampilan diperbarui secara otomatis ketika terjadi perubahan.

b. Penerapan prinsip SSoT pada informasi pengguna dilakukan dengan menyimpan objek User yang berisi atribut name dan memberStatus di dalam ViewModel melalui MutableStateFlow. Perubahan nama dan status membership hanya dilakukan menggunakan fungsi setMemberName() dan setMemberStatus(). Composable UserPanel menerima data tersebut melalui collectAsState() sehingga informasi pengguna selalu mengikuti kondisi terbaru yang tersimpan dalam ViewModel tanpa melakukan perubahan data secara langsung dari UI.

c. Percobaan 1 dan Percobaan 2 mematuhi prinsip Single Source of Truth karena pengelolaan nilai diskon dan informasi membership dilakukan secara terpusat di dalam ViewModel, sedangkan Composable hanya bertugas menampilkan data dan mengirimkan event dari pengguna. Perubahan state dilakukan melalui fungsi ViewModel, kemudian disalurkan ke UI menggunakan StateFlow dan collectAsState(). Pemisahan tanggung jawab tersebut membuat data lebih konsisten, mempermudah pemeliharaan kode, dan sesuai dengan pola arsitektur MVVM pada pengembangan aplikasi Android menggunakan Kotlin dan Jetpack Compose.

Percobaan 7 (7.5.2 Coroutine)

1.



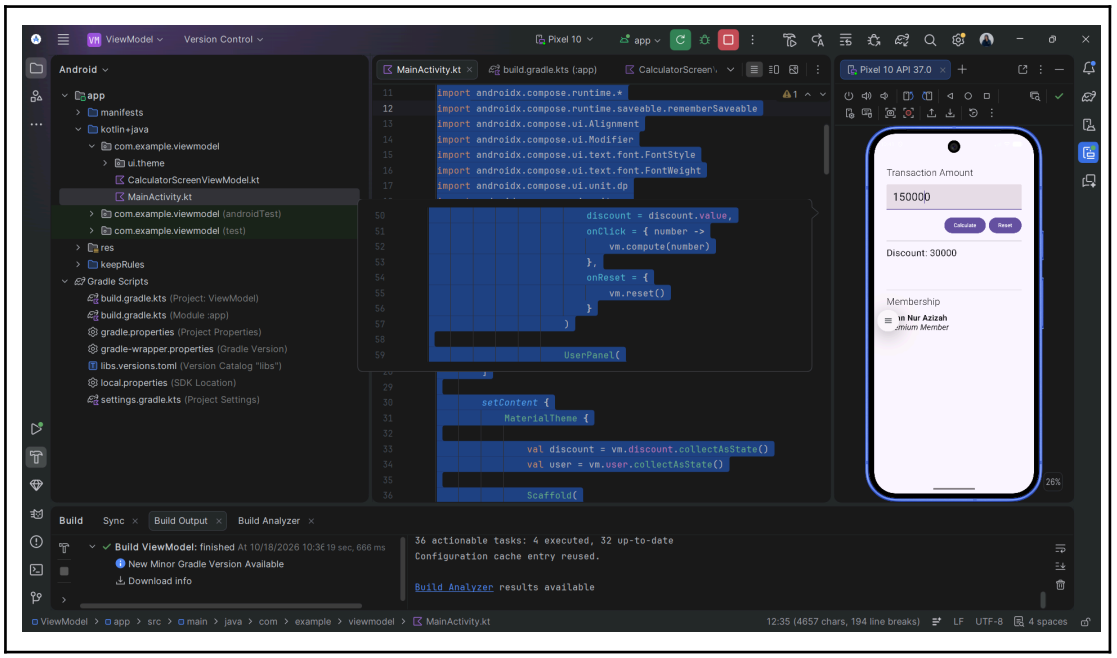
a. Setelah tombol Calculate diklik, antarmuka aplikasi menjadi tidak responsif selama proses perhitungan berlangsung. Pengguna tidak dapat langsung

mengubah nilai pada TextField atau menggunakan tombol Reset karena fungsi Thread.sleep(10000) memblokir Main Thread selama 10 detik.

b. Lima detik setelah tombol Calculate diklik, aplikasi masih dalam kondisi tidak responsif karena proses Thread.sleep(10000) belum selesai. Jika pengguna terus berinteraksi dengan aplikasi, sistem Android dapat menampilkan pesan Application Not Responding (ANR). Setelah waktu tunggu sekitar 10 detik berakhir, proses perhitungan dilanjutkan dan nilai diskon dapat diperbarui.

c. UI tidak dapat merespons karena fungsi compute() menjalankan Thread.sleep(10000) secara sinkron pada Main Thread. Main Thread bertanggung jawab untuk memproses interaksi pengguna dan memperbarui tampilan. Ketika thread tersebut diblokir selama 10 detik, aplikasi tidak dapat memproses event maupun melakukan rendering secara normal. Akibatnya, aplikasi terlihat membeku hingga proses selesai.

2. Penggunaan Coroutine

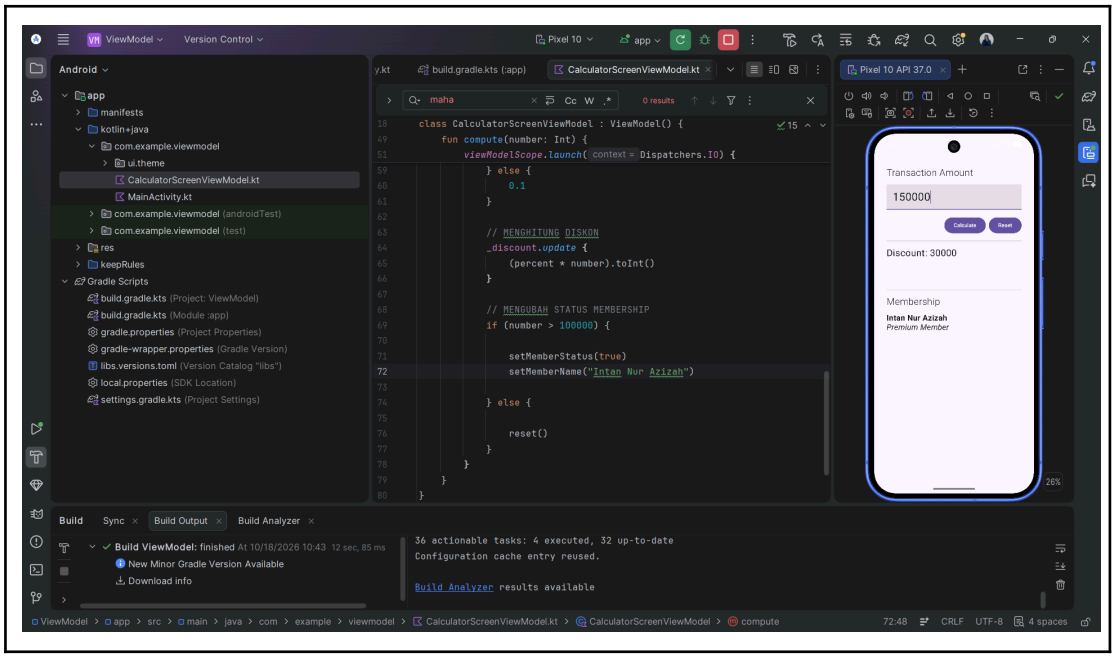


a. Setelah menggunakan Coroutine dengan viewModelScope.launch(Dispatchers.Default), aplikasi tetap dapat merespons interaksi pengguna selama proses perhitungan berlangsung. Nilai pada TextField masih dapat diubah dan tombol Reset tetap dapat diklik meskipun fungsi compute() membutuhkan waktu 10 detik. Hal ini terjadi karena proses perhitungan dijalankan pada thread background, sehingga tidak memblokir Main Thread.

b. Pada Nomor 1, proses perhitungan berjalan secara sinkron pada Main Thread sehingga aplikasi membeku selama 10 detik dan tidak dapat langsung merespons interaksi pengguna. Sedangkan pada Nomor 2, proses perhitungan dijalankan menggunakan Coroutine dengan Dispatchers.Default, sehingga Main Thread tetap tersedia untuk menangani interaksi dan memperbarui tampilan. Dengan demikian, aplikasi menjadi lebih responsif meskipun proses perhitungan masih berlangsung.

c. Pada pengujian Nomor 2, pesan Application Not Responding (ANR) seharusnya tidak muncul hanya karena proses perhitungan tersebut. Hal ini karena Thread.sleep(10000) dijalankan pada thread background menggunakan Dispatchers.Default, bukan pada Main Thread. Dengan demikian, aplikasi tetap dapat memproses event pengguna selama perhitungan berlangsung.

3. Setelah mengubah Dispatchers.Default menjadi Dispatchers.IO, aplikasi tetap dapat merespons interaksi pengguna meskipun proses perhitungan berlangsung selama 10 detik. TextField masih dapat diubah dan tombol Reset tetap dapat diklik tanpa menyebabkan antarmuka membeku. Tidak terdapat perbedaan tampilan yang signifikan dibandingkan penggunaan Dispatchers.Default, karena keduanya menjalankan proses pada thread background. Perbedaannya terletak pada fungsi utama masing-masing dispatcher, yaitu Dispatchers.Default digunakan untuk komputasi berat yang membutuhkan CPU, sedangkan Dispatchers.IO digunakan untuk operasi input/output atau pekerjaan blocking seperti akses database, file, dan jaringan.



4. Dispatchers.IO digunakan untuk menjalankan operasi input/output yang membutuhkan waktu, seperti mengambil data dari server melalui API, membaca atau menulis file, serta mengakses database. Contohnya pada aplikasi toko online, Dispatchers.IO digunakan untuk mengambil daftar produk dari server tanpa menghambat Main Thread, sehingga pengguna tetap dapat berinteraksi dengan aplikasi selama proses pengambilan data berlangsung.

```
viewModelScope.launch(Dispatchers.IO) {
    val products = repository.getProducts()
}
```

Dispatchers.Default digunakan untuk menjalankan proses komputasi yang membutuhkan banyak sumber daya CPU, seperti pengolahan gambar, perhitungan matematika kompleks, dan pengolahan data dalam jumlah besar. Contohnya pada aplikasi pengedit foto, Dispatchers.Default digunakan untuk menghitung efek filter pada gambar tanpa mengganggu respons antarmuka pengguna, sehingga aplikasi tetap lancar ketika proses pengolahan gambar berlangsung.

```
viewModelScope.launch(Dispatchers.Default) {
    val result = processImage(image)
}
```

Penggunaan Coroutine dengan Dispatchers.IO dan Dispatchers.Default membantu menjalankan pekerjaan di luar Main Thread agar antarmuka aplikasi tetap responsif. Dispatchers.IO lebih sesuai untuk operasi input/output, sedangkan Dispatchers.Default lebih sesuai untuk pemrosesan data yang membutuhkan CPU. Pemilihan dispatcher yang tepat dapat meningkatkan kinerja aplikasi serta memberikan pengalaman pengguna yang lebih baik.